CocoaMojo

Writing native macOS applications in Mojo, on a compiler that reads the SDK.

CocoaMojo is the Cocoa layer of MojoCocoa, an unofficial fork of Mojo frozen at one commit. It is two things at once: a small standard-library package, `std.objc`, and a compiler hook, `cocoakb`, that answers questions about macOS *while your program is being compiled*.

The result is that a Cocoa binding states a name and the compiler supplies everything else. A selector typo is a compile error. A struct that has drifted from the SDK fails to build. An argument in the wrong register file is a compile error rather than a corrupted call.

```
comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()

var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

These documents

Document	What it is
Programmer's Guide	Read this first. Nine chapters: the frozen dialect, Mojo's own object model, then Cocoa from the first message to a walked-through Life implementation.
Reference Manual	Look things up here. The frozen language dialect, every `std.objc` entry point, every `cocoakb` query, and every diagnostic.
GPU programming	Writing Mojo functions that run on the Apple GPU: the execution model, threads and memory, and how to build, run and verify them.

A warning about versions

Mojo has been changing quickly, and most writing about it — including parts of its own published documentation — describes a language this compiler does not accept. `@parameter if` and `@parameter for` have given way to `comptime if` and `comptime for`, and `alias` is deprecated in favour of `comptime`.

This fork has also stopped following upstream's declaration model and named itself **cocoa-mojo**. Two keywords upstream removed are back, with new and narrower meanings: **fn** is now the foreign-callable function — thin, non-raising, C ABI, exactly the Objective-C `IMP` contract — and **let** is an immutable, scope-bound binding. Neither means what it meant in older Mojo, and neither means what upstream would mean by it.

Everything in these documents was checked against the source tree of this frozen compiler, not against published documentation. Where the two disagree, the compiler is right and the documentation is stale. The Language reference lists the differences explicitly, because they are the single largest source of code that looks correct and will not build.

The state of the port

The Cocoa layer described here works and is verified: nine of nine verification spikes pass, and the example applications build and run.

The GPU layer is a working vertical slice: kernels compile, run and agree with a CPU reference, and 96 of 119 in-scope GPU tests pass, with the failures triaged and named. It has its own section, and nothing in the Cocoa chapters depends on it.

Conventions

Code shown in these documents is either taken directly from the fork's standard library and verification spikes, or is reduced from them. Where a listing is abbreviated, an ellipsis comment says so.

The database is a build input, not a runtime dependency. A compiled CocoaMojo program has no dependency on `cocoa.sqlite`. By the time code is generated, every query has already collapsed to a constant.

GPU programming in Mojo

Mojo's premise is that one source file specialises to whatever silicon you point it at, and the GPU is where that claim is cashed. A kernel is an ordinary Mojo function. There is no separate shading language, no string of source compiled at run time, and no second toolchain — you write Mojo, and the same compiler that produced your CPU code produces the GPU code.

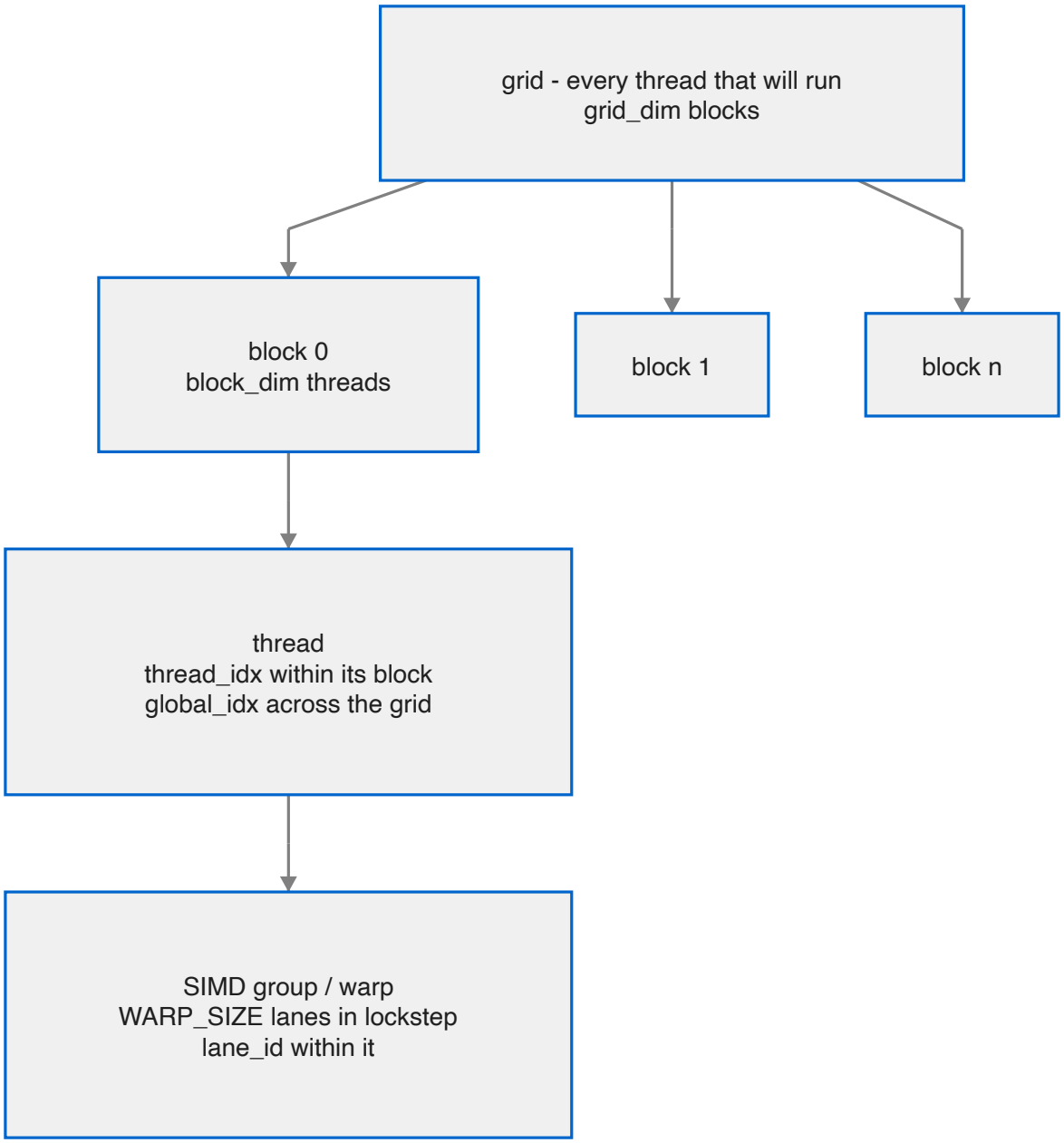
```
from std.gpu import global_idx
from max.gpu.host import DeviceContext

def scale(data: Pointer[Float32, MutAnyOrigin], factor: Float32, n: Int):
    var i = Int(global_idx.x)
    if i < n:
        data[unsafe_offset=i] = data[unsafe_offset=i] * factor
```

That is a complete GPU kernel. The rest of this section is about how to launch it, how to think about the threads that run it, and how to know the answer is right.

Document	What it covers
1. Your first kernel	The execution model, and a complete program from allocation to result
2. Threads, blocks and memory	Indexing, shared memory, barriers, and SIMD-group operations
3. Running and checking	Building, timing, verifying results, and what this hardware supports

The execution model, briefly

The same function body runs on thousands of threads at once. Each thread discovers *which* piece of work it owns by asking for its own index, does that piece, and stops. Threads are grouped into **blocks**, blocks make up the **grid**, and only threads within a block can cheaply cooperate.



If you have written CUDA or Metal compute, this is the model you already know, with Mojo's names on it. If you have not, the next chapter builds it up from a working program.

Where this runs

This fork targets the **Apple Silicon GPU** through Metal. The reference machine is an M4 Max. Kernels are ordinary Mojo, so the same source is what you would run on another vendor's hardware through a different backend — but on this machine, Metal is the backend and `DeviceContext(api="metal")` is how you ask for it.

1. Your first kernel

A kernel is a function

There is no kernel keyword and no attribute. A kernel is a `def` whose parameters can cross to the device, which asks for its own thread index and does one element's worth of work:

```
from std.gpu import global_idx

def mandelbrot_kernel(
    escape: Pointer[UInt32, MutAnyOrigin],
    center_x: Float32,
    center_y: Float32,
    scale: Float32,
):
    var idx = Int(global_idx.x)
    if idx < PIXELS:
        var px = idx % WIDTH
        var py = idx // WIDTH
        var cx = center_x + (Float32(px) - Float32(WIDTH) * 0.5) * scale
        var cy = center_y + (Float32(py) - Float32(HEIGHT) * 0.5) * scale

        var zx = Float32(0)
        var zy = Float32(0)
        var n = UInt32(0)
        while n < UInt32(MAX_ITER) and zx * zx + zy * zy <= Float32(4):
            var nzx = zx * zx - zy * zy + cx
            zy = Float32(2) * zx * zy + cy
            zx = nzx
            n += 1

        escape[unsafe_offset=idx] = n
```

Three things in that signature are worth pausing on.

Pointer[UInt32, MutAnyOrigin] — device buffers use `MutAnyOrigin`, not the `MutUntrackedOrigin` that Cocoa code uses. The pointer is tracked; it just has no single named origin.

Scalars pass by value. `center_x`, `scale` and friends are copied into the launch, so small parameters cost nothing to pass and need no buffer.

The bounds check is not optional. Grids are rounded up to whole blocks, so the final block runs threads past the end of your data. Without `if idx < n` those threads write outside the buffer.

The host side, in five calls

```
from max.gpu.host import DeviceContext

def main() raises:
    var ctx = DeviceContext(api="metal")
    var dev = ctx.enqueue_create_buffer[DType.uint32](PIXELS)
    var f = ctx.compile_function[mandelbrot_kernel]()
```

```
    comptime block = 256
    comptime grid = (PIXELS + block - 1) // block

    ctx.enqueue_function(f, dev, cx, cy, scale,
                         grid_dim=(grid), block_dim=(block))
    ctx.synchronize()
```

Call	What it does
`DeviceContext(api="metal")`	Opens the device
`enqueue_create_buffer[DType.T](n)`	Allocates n elements of device memory
`compile_function[kernel]()`	Compiles the kernel and builds its pipeline
`enqueue_function(f, args…, grid_dim=, block_dim=)`	Launches it
`synchronize()`	Waits for completion

`compile_function` is the expensive call. Hoist it out of any loop — compile once, launch many times.

Choosing the grid

```
comptime block = 256
comptime grid = (PIXELS + block - 1) // block
```

`block_dim` is how many threads are in each block; `grid_dim` is how many blocks. The ceiling division is the standard idiom for "enough blocks to cover `n` items", and it is exactly why the bounds check inside the kernel is required.

256 is a reasonable default block size. Both take tuples for 2D and 3D work — `grid_dim=(gx, gy)`, `block_dim=(16, 16)` — and then your kernel reads `global_idx.x` and `global_idx.y`.

Getting the answer back

Apple Silicon has unified memory, so reading results is a mapping rather than a copy:

```
with dev.map_to_host() as host_view:
    for i in range(PIXELS):
        ...host_view[i]...
```

It is a context manager, so the mapping is released at scope exit.

For explicit transfers — and for portability to discrete GPUs — buffers also carry `enqueue_copy_to`, `enqueue_copy_from` and `enqueue_fill`, and the context can allocate a host buffer with `enqueue_create_host_buffer`.

The whole program

Putting it together, with a CPU reference to check against:

```
from std.gpu import global_idx
from max.gpu.host import DeviceContext
from std.time import perf_counter_ns

comptime WIDTH = 1024
comptime HEIGHT = 768
comptime PIXELS = WIDTH * HEIGHT
comptime MAX_ITER = 256


def main() raises:
    comptime cx = Float32(-0.75)
    comptime cy = Float32(0.0)
    comptime scale = Float32(3.0) / Float32(WIDTH)

    # CPU reference
    var cpu_list = List[UInt32](length=PIXELS, fill=0)
    var cpu_buf = cpu_list.unsafe_ptr().unsafe_origin_cast[MutAnyOrigin]()
    mandelbrot_cpu(cpu_buf, cx, cy, scale)

    # GPU
    var ctx = DeviceContext(api="metal")
    var dev = ctx.enqueue_create_buffer[DType.uint32](PIXELS)
    var f = ctx.compile_function[mandelbrot_kernel]()
    comptime block = 256
    comptime grid = (PIXELS + block - 1) // block

    ctx.enqueue_function(f, dev, cx, cy, scale,
                         grid_dim=(grid), block_dim=(block))
    ctx.synchronize()

    var agree = 0
    with dev.map_to_host() as gpu_buf:
        for i in range(PIXELS):
            if gpu_buf[i] == cpu_buf[i]:
                agree += 1
    print("agreement:", Float64(agree) / Float64(PIXELS) * 100.0, "%")
```

On the reference M4 Max the most recent run of this reports:

```
GPU: 0.413 ms   speedup: 197.96 x
exact agreement: 100.0 % ( 0 boundary-band pixels differ)
```

Before you take that as a benchmark, read Running and checking — both the timing and the comparison need more care than they look like they do.

Running it

From the CocoaMojo distribution, one command either way:

```
cocoamojo --run   mandelbrot.mojo      # JIT: compile and run
cocoamojo --build mandelbrot.mojo      # -> ./mandelbrot
```

Both work for GPU code. `--run` JITs the program, GPU kernels included, and `--build` produces an ordinary double-clickable binary.

No `-I` flags, no `MODULAR_*` variables, no Bazel — the distribution carries the compiler, the runtimes, the packages and the SDK database beside each other, and `cocoamojo` wires them up. Chapter 3 covers the details, including what to do when you are building against the source tree rather than a distribution.

2. Threads, blocks and memory

Chapter 1 used one index and one buffer. Real kernels need to know where they sit in the grid, cooperate with their neighbours, and use faster memory than the device buffer. This chapter is those three things.

Knowing where you are

```
from std.gpu import (
    thread_idx, block_idx, block_dim, grid_dim, global_idx,
    lane_id, warp_id, WARP_SIZE,
)
```

Name	What it tells you
`thread_idx.x/.y/.z`	This thread's position **within its block**
`block_idx.x/.y/.z`	This block's position within the grid
`block_dim.x/.y/.z`	How many threads per block
`grid_dim.x/.y/.z`	How many blocks in the grid
`global_idx.x/.y/.z`	Position across the whole grid
`lane_id`	This thread's lane within its SIMD group
`warp_id`	Which SIMD group within the block
`WARP_SIZE`	Lanes per SIMD group

`global_idx` is the convenience you will use most; it is `block_idx * block_dim

- thread_idx` computed for you.

For 2D work, index both axes:

```
def blur(out: Pointer[Float32, MutAnyOrigin], w: Int, h: Int):
    var x = Int(global_idx.x)
    var y = Int(global_idx.y)
    if x < w and y < h:
        var i = y * w + x
        ...
```

launched with `grid_dim=(gx, gy), block_dim=(16, 16)`.

A note on `WARP_SIZE`

Do not hardcode it. Apple SIMD groups are 32 lanes; AMD's wave64 is 64. `WARP_SIZE` is a compile-time constant for the target you are building for, and a kernel that assumes 32 or 64 is a kernel that silently produces wrong answers on the other vendor. This is one of the commonest portability bugs in GPU code and it costs nothing to avoid.

Shared memory

Every thread can reach the device buffers you passed in, but that memory is comparatively slow. Threads *within a block* also share a small, fast scratchpad — Metal calls it threadgroup memory, CUDA calls it shared memory — and using it well is most of what separates a fast kernel from a slow one.

```
from std.memory import unsafe_stack_allocation, AddressSpace

comptime TILE = 16

def tiled_matmul(...):
    var a_shared = unsafe_stack_allocation[
        TILE * TILE, Float32, address_space=AddressSpace.SHARED,
    ]()
    var b_shared = unsafe_stack_allocation[
        TILE * TILE, Float32, address_space=AddressSpace.SHARED,
    ]()
```

The allocation is per block, and its size must be a compile-time constant. Ask for too much and the pipeline will not create.

The classic use is tiling: each block cooperatively loads a tile of input into shared memory, then every thread in the block reads that tile many times without touching device memory again.

Barriers

Shared memory is only useful if the threads agree about when it is filled. That is what a barrier is for:

```
from max.gpu.sync import barrier

    a_shared[unsafe_offset=local] = a[unsafe_offset=global_a]   # load
    barrier()                                                   # all loaded
    ...read a_shared freely...
    barrier()                                                   # all finished
```

`barrier()` synchronises **every thread in the block** — it is CUDA's `__syncthreads()`. Memory operations before it are visible to all threads after it.

Two rules that are not negotiable:

Every thread in the block must reach every barrier. Putting one inside an `if` that only some threads take is a hang or worse, not an error message. If you need a conditional load, do the condition *inside* the guarded region and put the barrier outside it.

You need the second barrier too. Without one before the next tile is loaded, a fast thread can overwrite shared memory a slow thread is still reading.

`syncwarp(mask)` synchronises only within a SIMD group, which is cheaper and occasionally what you want.

SIMD-group operations

Threads in the same SIMD group run in lockstep and can exchange values directly — no shared memory, no barrier. This is the fastest cooperation available.

```
from std.gpu.primitives.warp import (
    shuffle_idx, shuffle_up, shuffle_down, shuffle_xor,
    reduce, lane_group_reduce,
)
```

Operation	What it does
`shuffle_idx`	Read another lane's value by absolute lane index
`shuffle_up` / `shuffle_down`	Read from a lane a fixed distance away
`shuffle_xor`	Read from the lane whose id differs by a mask — the butterfly pattern
`reduce`	Combine a value across the whole SIMD group
`lane_group_reduce`	The same across a sub-group of lanes

The idiom worth knowing is the reduction. To sum across a SIMD group, halve the distance each step:

```
    var v = my_value
    var offset = WARP_SIZE // 2
    while offset > 0:
        v += shuffle_down(v, offset)
        offset //= 2
    # lane 0 now holds the sum
```

A block-wide reduction is this, then one value per SIMD group written to shared memory, a barrier, and one more SIMD-group reduction over those.

Which memory to reach for

Scope	Reach	Cost	Use for
Registers	one thread	fastest	everything, by default
SIMD-group shuffles	32 lanes	very fast	reductions, scans, neighbour exchange
Shared memory	one block	fast	tiles, staging, block-wide cooperation
Device buffers	whole grid, and the host	slow	inputs and outputs

The usual shape of a fast kernel: read device memory once, coalesced; keep the working set in shared memory and registers; cooperate through shuffles where possible and barriers where not; write device memory once.

Coalescing

When adjacent threads read adjacent addresses, the hardware services them as one wide transaction. When they stride, it cannot. This is usually the difference between a kernel that is memory-bound at a sensible fraction of bandwidth and one that is ten times slower for no visible reason.

In practice: map the **fastest-varying thread index to the fastest-varying data index**. If your matmul reads `B[k][n]`, let `thread_idx.x` walk `n`, not `k`. The tiled matmul in this tree carries exactly that comment — *map thread x to column for coalesced access in B*.

Divergence

Threads in a SIMD group share one instruction pointer. When they take different branches, both sides execute with the inactive lanes masked off, so a two-way branch inside a SIMD group costs the sum of both sides.

Divergence across *different* SIMD groups is free. So a condition on `block_idx`, or on `global_idx / WARP_SIZE`, costs nothing; a condition on `lane_id` costs. Data-dependent branches — like the Mandelbrot escape loop — inevitably diverge, and that is fine; it is worth knowing rather than worth avoiding at all costs.

3. Running and checking

Running: JIT or build

From a CocoaMojo distribution:

```
cocoamojo --run   prog.mojo        # JIT, straight into this process
cocoamojo --build prog.mojo        # -> ./prog, a normal binary
cocoamojo --build prog.mojo -o app # -> ./app
```

Both paths run GPU kernels. `cocoamojo` is the whole interface: no `-I` flags, no `MODULAR_*` environment variables, no Bazel. The distribution ships the compiler, the Mojo runtime, the Metal device runtime (`libCocoaMojoGPU`), the packages and `cocoa.sqlite` beside one another, and the driver points the compiler at them. Binaries from `--build` carry an rpath to `lib/`, so they run anywhere on the machine without a wrapper.

A correction worth knowing

Earlier versions of this fork's documentation — including earlier versions of this guide — said that GPU code could not be JIT-run, because "the JIT cannot resolve the GPU runtime's symbols". **That was wrong**, and the mistake is worth understanding because it is a common shape.

The JIT was never the problem. `ExecutionEngineOptions::libraryPaths` feeds an ORC dynamic-library search generator, and `-Xlinker -L`/`-l` reach it. The symbols were missing for the same reason so many things here were missing: nothing exported them. Once the device runtime existed as `libCocoaMojoGPU.dylib` rather than a hidden-visibility archive, the JIT resolved them like any other library and ran GPU kernels immediately:

```
GPU: 0.413 ms   speedup: 197.96 x
exact agreement: 100.0 % ( 0 boundary-band pixels differ)
COMPUTE-SMOKE: PASS
```

All sixteen spike checks now pass through the JIT path.

The lesson generalises: *"the JIT cannot do X"* is almost always *"nothing exported the symbol X needs"*, and the two have very different fixes.

One reason to prefer a subprocess anyway

The argument for building and exec'ing does survive, in one specific form. JIT'd code runs in the **host process's address space**. A segfault is caught by `CrashRecoveryContext`, but a call to `exit()` is not, and it would take the host down with it.

That matters if you are embedding the compiler in something long-lived — an editor, say. For running your own programs from a shell it is irrelevant.

Building against the source tree

If you are working in the repository rather than from a distribution, you drive the compiler directly and supply what `cocoamojo` would have supplied:

```
mojo build --target-accelerator apple-m4 \
  -I mojo/stdlib -I max/kernels/src -I max/mojo \
  -o /tmp/prog prog.mojo
```

`--target-accelerator` is required

Through Bazel the toolchain supplies it. Compiling directly you must pass it yourself, or comptime GPU dispatch fails with *"Unknown GPU architecture"*.

Clear the kernel cache when a change seems not to take

Compiled kernels are cached, so a rebuilt compiler can still hand you yesterday's answer. When an edit appears to do nothing, suspect this first:

```
./clear_cache.sh
```

`./rebuild.sh` does it as part of a rebuild, and can assert that a marker string is genuinely present in the binary you are about to run — so "did it actually rebuild?" has an answer that is evidence rather than hope.

One Bazel trap worth knowing: **Bazel does not forward your shell environment into compile actions**, so exporting a variable does nothing. Use `--action_env=FOO=1`, which forwards it *and* changes the action key so the action re-runs — at the cost of invalidating the whole graph, so expect a full rebuild each way.

Timing

Warm up first. The first dispatch pays for pipeline creation and first-touch residency, and including it will make your kernel look several times slower than it is.

```
    # warm
    ctx.enqueue_function(f, ...); ctx.synchronize()

    var t0 = perf_counter_ns()
    ctx.enqueue_function(f, ...); ctx.synchronize()
    var t1 = perf_counter_ns()
```

`synchronize()` before reading the clock, or you are timing the enqueue rather than the work.

Launch is synchronous by default. Setting `APPLEGPU_ASYNC_LAUNCH=1` defers the wait to `synchronize()`, which is worth about +29% on a single-chain FMA kernel and +2.9% at 64 chains — most valuable when you are launch-bound and nearly irrelevant when you are not.

Checking the answer

This deserves more care than it usually gets, because the obvious check is the wrong one.

Comparing every GPU value to every CPU value for exact equality will fail on correct code. The GPU contracts a multiply and an add into a single fused operation; the CPU does them separately, rounding in between. For most data the results are identical, but wherever the computation is iteration-sensitive the two can diverge.

The Mandelbrot spike says it plainly:

> CPU and GPU agree exactly on interior and exterior pixels. In the thin chaotic band right at the set boundary, a point is so iteration-sensitive that the GPU's fused multiply-add (vs the CPU's separate mul/add) can shift its escape count — inherent to Float32 mandelbrot, not a bug.

So it measures an **agreement rate** and requires it to be very high:

```
var rate = Float64(agree) / Float64(checked) * 100.0
print("PASS" if rate > 99.0 else "FAIL")
```

The general rule is to **classify differences before claiming victory or defeat**:

Difference	Verdict
Boundary of a chaotic region, tiny population	arithmetic — expected
Last-bit differences in float output	contraction — expected
Smooth, low-iteration region	codegen bug
Large, structured, or in a flat area	codegen bug
Exactly one block's worth of values wrong	a bounds check or an index
Everything past a certain index wrong	grid too small, or a missing `if i < n`

A picture that looks right is not evidence. A picture that survives a classification filter is.

Debugging

The runtime reads a set of environment variables, and the two you will reach for first are:

Variable	Use
`APPLEGPU_TRACE_LAUNCH`	Trace each dispatch — confirms the launch happened and with what dimensions
`APPLEGPU_TRACE_CAPS`	Report the device capabilities resolved at context creation

`APPLEGPU_KEEP_AIR` keeps the intermediate GPU module for inspection, and `APPLEGPU_TRACE_BLOB`, `APPLEGPU_TRACE_PROFILE` and `APPLEGPU_AIR_TRACE_KNOBS` cover capture blobs, timing and which knobs are in effect.

Metal's own validation is worth enabling during bring-up: this fork's dispatch path passes with Metal debug and shader validation on, and a residency mistake that is silent otherwise becomes an error there.

What this hardware supports

The reference machine is an **M4 Max**, and one distinction shapes what runs.

Apple's 16×16 `simdgroup_matrix` operations — the tensor-core-class matrix paths — require `MTLGPUFamilyApple10`, which **M5 is the first part to advertise**. On an M4 those kernels compile, and the module loads, and the driver then declines the pipeline with *"supported by GPUFamily10 and later"*.

You will meet this as a runtime error mentioning Apple M5, not as a compile failure, because everything up to pipeline creation succeeds. Guards in the kernel library are spelled `compute_capability() != 5`, and this fork answers that question by asking Metal for the family rather than trusting a marketing name.

The 8×8 matrix path does work here.

An honest word on maturity

The GPU support is a **working vertical slice**, not a finished backend. A Mandelbrot runs and agrees with the CPU; 96 of 119 in-scope GPU tests pass; the failures are triaged and named. The open work is mostly numerical correctness in specific kernels, plus the M5 surface that cannot be exercised on this machine at all.

Two consequences for you. Ordinary kernels of the shape in this section — index, guard, compute, store — are well-travelled ground. The further you go towards the kernel library's specialised paths, the more likely you are to meet something that has not been validated here yet, and the more the advice is to verify numerically against a CPU reference rather than to assume.

CocoaMojo Programmer's Guide

This guide teaches CocoaMojo in the order you meet it: get a compiler running, learn what the language actually is at this frozen version, understand the object model Mojo has on its own, then send your first message to Cocoa, get memory right, let Cocoa call back into your code, and finally assemble and read a real windowed application.

Read it in order the first time. Each chapter assumes the one before it.

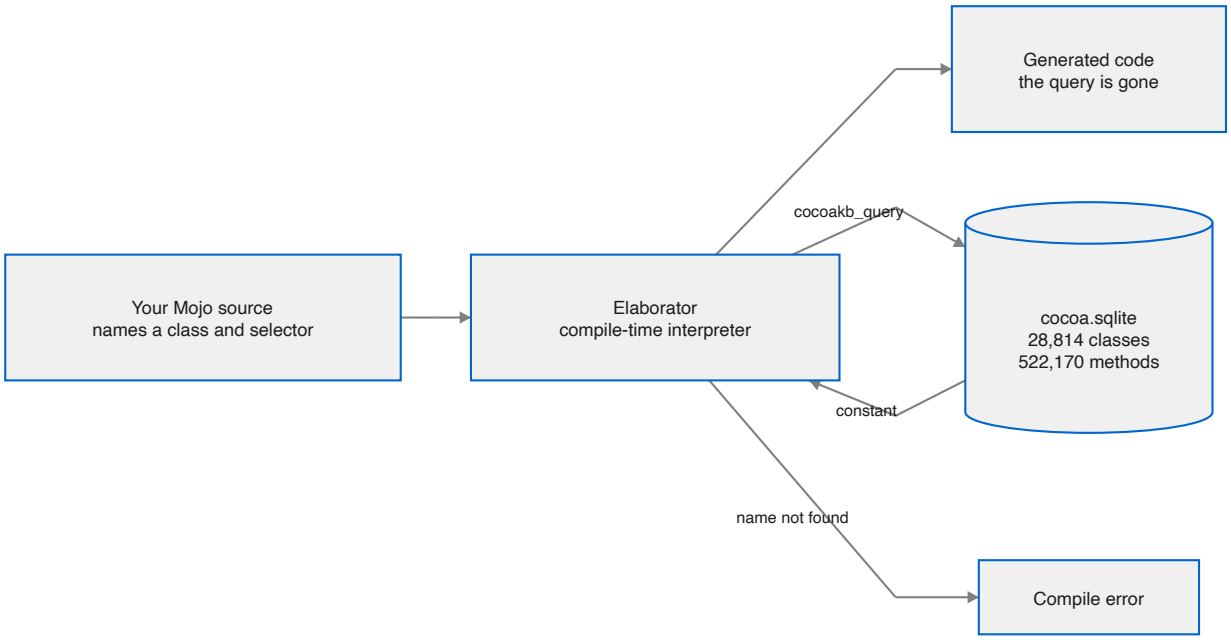
Chapter	What you will be able to do
1. Getting started	Build the compiler, build the SDK database, compile and run a program
2. The language, as it is now	Write code this compiler accepts, and recognise the constructs it rejects
3. Mojo's own object model	Structs, traits, generics and lifetimes — what Mojo gives you before any Cocoa appears
4. Calling Cocoa	Look up classes, send messages, pass arguments, get results back
5. Ownership and memory	Hold Cocoa objects without leaking and without over-releasing
6. Letting Cocoa call you	Declare Objective-C classes with `class`, so Cocoa can send your code messages
7. A complete application	Put a window on screen, handle events, and drive a run loop
8. Concurrency and blocks	Dispatch work across GCD queues, and call block-only Cocoa APIs
9. A demo, walked through	Read Conway's Life — a complete 644-line Cocoa application — and see how little of it is Cocoa

When you want to look something up rather than learn it, use the Reference Manual.

What makes this different from a binding layer

Most language bridges to Cocoa work by generating declarations: a tool reads the SDK once, emits a large body of source, and from that moment the generated code and the real SDK drift apart silently.

CocoaMojo does not generate anything. The compiler holds an open connection to a database describing macOS, and resolves each fact at the point of use during elaboration.



Two consequences follow, and they are the reason the whole design exists.

The first is that **a wrong name cannot survive to run time**. There is no generated stub to be out of date, so a selector the SDK does not have has nowhere to hide.

The second is that **the compiled program has no dependency on the database**. By the time code is generated, every query has collapsed to a constant. The database is a build input in exactly the way a header file is.

1. Getting started

What you need

Requirement	Why
Apple Silicon Mac	The fork targets `arm64` Darwin and hardcodes it deliberately
macOS 15 or later	The metadata is built from the live Objective-C runtime on your machine
Xcode 16 or later	Supplies the SDK and, if you go near the GPU, the AIR toolchain
A checkout of MojoCocoa	The compiler and standard library
A checkout of CocoaBaseMCP	Builds `cocoa.sqlite`

The two checkouts are expected to sit beside each other. Everything below assumes that layout; if yours differs, one environment variable fixes it.

Build the SDK database

The database is built from *your* machine — the live Objective-C runtime plus BridgeSupport — not downloaded. It takes about twelve seconds.

```
python3 ../CocoaBaseMCP/build.py
```

You get roughly 236 MB describing 28,814 classes and 522,170 methods.

Rebuild it after any macOS update. If you do not, the compiler will happily keep using the old one, and `cocoakb_db_hash()` is how you notice: it returns the SHA-256 of the database a given compilation consulted, so a binary can record exactly which revision it was built against.

Build the compiler

```
./bazelw build --config=build-mojo //KGEN:mojo
```

This is a long build the first time. Afterwards, only toolchain or sysroot changes force a full rebuild — ordinary source changes rebuild KGEN alone, in seconds rather than the better part of an hour.

The compiler lands at `bazel-bin/KGEN/tools/mojo/mojo-full`.

Point the compiler at the database

```
export MODULAR_MOJO_MAX_COCOAKB_PATH="$PWD/../CocoaBaseMCP/cocoa.sqlite"
```

`local.bazelrc` sets this for Bazel-driven builds. Export it yourself when invoking the compiler directly.

Run a program

There are two ways in, and which you want depends on whether you are *using* CocoaMojo or *working on* it.

From a distribution

`cocoamojo` is the whole interface — no `-I` flags, no `MODULAR_*` variables, no Bazel:

```
cocoamojo --run   hello.mojo         # compile and run
cocoamojo --build hello.mojo         # -> ./hello
cocoamojo --build hello.mojo -o app  # -> ./app
```

The distribution carries the compiler, the Mojo runtime, the Metal device runtime, the packages and `cocoa.sqlite` beside one another, and the driver wires them together. `--run` JITs; both paths work for Cocoa and for GPU code. Binaries from `--build` carry an rpath to `lib/`, so they run anywhere on the machine without a wrapper.

From the source tree

Working in the repository there is a wrinkle worth knowing before it wastes your afternoon: the raw `mojo-full` binary cannot locate `std.mojoc` on its own. Run Mojo through the Bazel wrapper, which supplies the standard-library import paths:

```
./bazelw run //KGEN:mojo -- run /absolute/path/to/program.mojo
```

Note the **absolute** path. Bazel runs the wrapper in its own working directory, so a relative path will not resolve the way you expect.

Your first program

Save this as `hello_cocoa.mojo`:

```
from std.objc import ObjCClass, ObjCObject, msg_send, autoreleasepool
from std.ffi import c_char
from std.memory import OpaquePointer


def main():
    with autoreleasepool():
        var cls = ObjCClass.lookup["NSString"]()
        var text = String("Hello from Cocoa, via Mojo")

        var s = msg_send[
            ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
        ](cls.as_object(), text.as_c_string_slice())

        var n = msg_send[Int, "NSString", "length"](s)
        print("length:", n)
```

```
        var back = msg_send[
            OpaquePointer[MutUntrackedOrigin], "NSString", "UTF8String"
        ](s)
        print("round trip:", String(unsafe_from_utf8_ptr=back.bitcast[c_char]()))
```

Run it, and you should see:

```
length: 26
round trip: Hello from Cocoa, via Mojo
```

Nothing in that program named a dispatch function, a type encoding, or a selector address. Those all came from the database while the program was being compiled.

Prove the checking is real

The interesting half of the verification suite is the half that must *fail*. Change `"length"` to `"lenght"` and compile again. You do not get a runtime crash or a silently wrong answer; you get a compile error naming the selector.

The fork ships this as a test:

```
./spikes/run-cocoa-checks.sh
```

It runs twelve spikes that must compile and run, and four that must be **rejected at compile time** — sixteen checks in all. A run in which the must-fail spikes quietly succeed is a failed run, because the design's entire claim is that unknown names cannot reach code generation.

The four must-fail checks are worth knowing individually, because each pins one guarantee: an unknown metadata name (`must_fail.mojo`), a wrong argument count (`must_fail_argcount.mojo`), a raising `fn` (`must_fail_fn_raises.mojo`), and reassigning a `let` (`must_fail_let_assign.mojo`).

Where to go next

Chapter 2 is not optional. This compiler rejects a good deal of Mojo that current tutorials still teach, and knowing what changed will save you more time than anything else in this guide.

2. The language, as it is now

This chapter exists because most Mojo you will find written down does not compile here, for two separate reasons. Mojo itself moved and the writing did not; and this fork has since started changing the language deliberately, under the name cocoa-mojo.

Read this before you write anything substantial. Everything below was taken from the compiler's own source and standard library at the frozen commit.

This is cocoa-mojo, not upstream Mojo

The fork has stopped tracking upstream's declaration model and named itself. Where upstream removed something this port needs, it heals the language instead of working around the removal. Two keywords have come back with new, narrower meanings, and they are the first thing to learn.

The stance is recorded in the fork's own COCOA_LET_DESIGN.md: new code here is deliberately a different dialect, and migration diagnostics teach rather than promise compatibility.

`fn` is back — as the foreign-callable function

Upstream removed `fn`. This fork revived it, but not as the old strict function. Here `fn` declares a **foreign-callable** function:

- **thin** — no captured context
- **non-raising** — the C boundary has no error channel
- **C ABI** — set automatically by the keyword
- every parameter and return type must be ABI-classifiable

That is exactly the contract an Objective-C `IMP` requires, which is the point.

```
fn add_one(x: Int) -> Int:
    return x + 1

fn button_clicked(self_: P, cmd: P, sender: P):
    clicks()[] += 1
```

You do not write `abi("C")` any more. The keyword says it.

An `fn` is still an ordinary callable from Mojo — `add_one(1)` works — so the keyword costs you nothing at the call site.

`fn` in type position

`fn(...) -> R` is sugar for the old `def(...) thin abi("C") -> R`:

```
comptime IntFn = fn(Int) -> Int

def apply(f: IntFn, x: Int) -> Int:
    return f(x)
```

The standard library's IMP aliases are now spelled this way, and read as what they are:

```
comptime IMP0 = fn(P, P, /) -> None
comptime IMP1 = fn(P, P, P, /) -> None
comptime IMP0Bool = fn(P, P, /) -> Bool
comptime IMP1Bool = fn(P, P, P, /) -> Bool
comptime IMP2 = fn(P, P, P, P, /) -> None
```

What `fn` rejects

Marking one `raises` is a compile error, and the message teaches rather than just refusing:

```
'fn' declares a foreign-callable (C ABI, non-raising) function in cocoa-mojo
and may not be marked 'raises'; use 'def' for an ordinary Mojo function
```

It comes with a FixIt that replaces `fn` with `def`. `async` is rejected the same way. Old code of the `fn main() raises` shape therefore gets told what changed instead of a bare contract violation.

`let` is back — as the immutable binding

`let` declares an **immutable, scope-bound binding**.

```
let win = ObjCObject(window_addr()[])
let n = 41 + 1
```

Three things to know.

The binding is immutable; the object is not. This is Swift's rule exactly. You cannot rebind `win`, but you can send it setters all day.

It is general. The design originally proposed gating `let` on a `Retained` trait so that only reference-counted Cocoa objects could use it. When it came to implementation the opposite was chosen: `let n = 42` and `let win = NSWindow(...)` both mean "immutable binding", which is one rule instead of two.

Ownership rides the value, not the keyword. For a Cocoa object held in an `ObjCRef`, `let` gives you the same +1-at-bind and release-at-scope-exit you already had with `var`; the ARC behaviour lives in `ObjCRef`. `let` adds only the immutability of the binding.

What `let` rejects

Reassignment is a compile error. And `let` is a **function-body** form only — there is no field or file-scope `let`:

```
'let' declares an immutable binding inside a function body; use 'var' for a
field or module value
```

When to reach for it

The fork's own conversion is a good guide: sixteen never-reassigned Cocoa bindings became `let` in one file alone. If you never rebind it, say so.

`alias` is deprecated, not removed

`alias` still parses, with a warning:

```
'alias' is deprecated; use 'comptime'
```

Use `comptime`. `alias` is gone from the standard library's own code.

`comptime` does the compile-time work

Argument conventions

There is no `inout`, no `borrowed`, and no `owned`. The conventions are:

Most of them are *contextual* — soft identifiers rather than reserved words — so `mut`, `imm`, `out` and `deinit` are all still available as ordinary variable names. Only `var` is a reserved keyword.

Convention	Meaning
(none)	Borrowed immutably. The default; you do not write it.
`imm`	Borrowed immutably, said out loud. `read` is the deprecated spelling.
`mut`	Borrowed mutably.
`out`	An uninitialised result the function must initialise. Used for `__init__`.
`var`	Taken by value; the callee owns it.
`deinit`	Consumes the value and ends its lifetime. Used for `__deinit__` and for consuming methods.

In practice:

```
def __init__(out self, *, adopt: ObjCObject):
    self._obj = adopt

def _add(mut self, selector: StaticString, encoding: String, imp: P):
    ...

def register(deinit self) -> ObjCClass:
    ...

def __iter__(var self) -> Self.IteratorOwnedType:
    ...
```

The transfer sigil `^` moves a value into a consuming position:

```
var cls = b^.register()      # b is consumed here
var delegate = new_instance(cls)
```

Forgetting `^` on a `deinit` method is one of the more common early errors.

Origins, and how they are spelled

References carry an origin parameter. The names were renamed twice and the old spellings have now been removed, so a tutorial written a few months ago will use identifiers that no longer exist.

Use this	Not this
`ImmOrigin`	`ImmutOrigin`
`ImmUnsafeAnyOrigin`	`ImmutUnsafeAnyOrigin`
`ImmStaticOrigin`	`StaticConstantOrigin`
`UntrackedOrigin`	`ExternalOrigin`
`MutUntrackedOrigin`	`MutExternalOrigin`
`ImmUntrackedOrigin`	`ImmutUntrackedOrigin`, `ImmutExternalOrigin`

`MutUntrackedOrigin` is the one you will type constantly in Cocoa code, because every pointer that crosses into Objective-C is outside Mojo's lifetime tracking by definition. Most CocoaMojo files start by shortening it:

```
comptime P = OpaquePointer[MutUntrackedOrigin]
```

Pointers

`Pointer` is the type. `UnsafePointer` still exists but is deprecated in favour of it, and the pre-unification aliases — `MutUnsafePointer`, `ImmUnsafePointer`, `ImmutOpaquePointer`, `OptionalUnsafePointer` and the rest — have been removed outright.

```
var p = Pointer[UInt8, MutUntrackedOrigin](unsafe_from_address=addr)
var q = OpaquePointer[MutUntrackedOrigin](unsafe_from_address=addr)
```

The raw memory functions were renamed to carry an `unsafe_` prefix, and the old names are gone: use `unsafe_memcpy`, `unsafe_memset`, `unsafe_memset_zero`, `unsafe_destroy_n`. Likewise the `Pointer` methods are now `unsafe_deinit_pointee()`, `unsafe_write()`, `unsafe_as_noalias()`.

Function types, `thin`, and `abi("C")`

`fn` is the spelling you want for a foreign-callable function, but the longhand still exists and you will meet it in older code and in diagnostics.

```
comptime IMP1 = fn(P, P, P, /) -> None          # cocoa-mojo
comptime IMP1 = def(P, P, P, /) thin abi("C") -> None   # the longhand it sugars
```

`/` ends the positional-only parameters, `thin` means the function carries no captured context, and `abi("C")` gives it the C calling convention. You need all three for a Cocoa callback, because the Objective-C runtime calls a bare C function pointer with no closure environment — which is precisely why `fn` bundles them into one keyword.

A `thin` function type can also carry trailing `where` clauses constraining the parameters it declares, so a generic algorithm can state what it requires of a function handed to it rather than restating the constraint at every binding site.

One consequence worth knowing: **a thin fn is exactly a global block.** An Objective-C block with no captures is `_NSConcreteGlobalBlock` with no copy/dispose helpers, so the standard library can build a block around any `fn` at no cost. That is how block-only Cocoa APIs are reachable today — see Concurrency and blocks.

Parameters and generics

Parameters go in square brackets and are resolved at compile time. Arguments go in parentheses.

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R:
    ...
```

That signature shows most of what you need: a type parameter, string parameters, a defaulted `Bool` parameter, a variadic type-parameter pack, and a value pack `*args: *Ts` bound to it.

Overloading works on parameter lists as well as argument lists, which is how `ObjCClassBuilder.add_method` accepts five different IMP shapes under one name.

Structs and traits

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64
```

`@fieldwise_init` synthesises the memberwise initialiser. Conformances go in the parentheses. The ones you will meet most in Cocoa code are `Copyable`, `Movable`, `AnyType` and `TrivialRegisterPassable`.

Two more renames that will bite: `ImplicitlyDestructible` and `ImplicitlyDeletable` are now `Deinitable`, and `@explicit_destroy` is no longer required, because implicit deletion is the default.

Destructors are `__deinit__`, taking `deinit self`:

```
def __deinit__(deinit self):
    if not self._obj.is_nil():
        ...
```

Other removals you are likely to trip over

Gone	Use instead
`InlineArray`	`Array`
`SIMD.size`, `Array.size`, `SIMDSize`	`length`, `SIMDLength`
`String.as_string_slice()`	`StringSpan(my_string)`
`as_immutable()`, `get_immutable()`	`as_imm()`
`ImmutSpan`	`ImmSpan`
`ConditionalType`	the ternary `T if cond else U`
`trait_downcast()`	a `where` clause or `comptime assert` with `conforms_to(...)`
`.mojopkg` files	`.mojoc` files
`memcmp`	`unsafe_memcmp`
`List.steal_data()`	`unsafe_take_allocation()`
`OwnedPointer.take()`	`into_inner()`
`Variant.take()`	`unwrap()`
`std.gpu.profiler` / `ProfileBlock`	`perf_counter_ns()`, or a real GPU profiler

The module system also tightened. Importing same-named functions from different modules to build one overload set is now an error, and intra-package access without an explicit `import` is an error. Both had deprecation periods that have now expired.

`async` on an `fn`

`fn` cannot be `async`; use `def`. The rest of the async story is unchanged from upstream, and nothing in CocoaMojo depends on it.

A note on `async`

`AnyCoroutine`, `Coroutine` and `RaisingCoroutine` are no longer in the prelude and the module defining them is private. `async def` still works and the compiler still synthesises those types — you simply cannot name them. Mojo's async support is unfinished and nothing in CocoaMojo depends on it.

What you can rely on

The point of the freeze is that this list will not move under you. Code written against this chapter keeps compiling for as long as you keep the compiler. That is the trade the fork makes: no upstream fixes and no new features, in exchange for a language that stays still long enough to build something on.

3. Mojo's own object model

Before any Cocoa appears, it is worth being clear about what Mojo gives you on its own — because it is not what most object-oriented experience will lead you to expect, and because the shape of the Cocoa layer is a direct response to the gap.

Mojo has no classes of its own. It has structs with methods, traits, and generics, and everything is resolved at compile time. There is no inheritance, no vtable, no runtime object graph, and no dynamic dispatch anywhere in the language.

That is not a deficiency to work around. Most of what you want from objects is here; what is missing is precisely the part Objective-C is unusually good at, which is why the two compose so well.

`class` means something specific here

In this fork `class` is not a Mojo class. It declares an **Objective-C** class — a real one, registered with the ObjC runtime, with dynamic dispatch and inheritance and everything else Mojo does not have.

That is the division of labour in one keyword. `struct` is Mojo's value type, resolved statically; `class` is Cocoa's reference type, resolved by the runtime. They are different animals and the language now says so.

This chapter is about `struct`, because that is what Mojo gives you on its own and what everything else is built from. Chapter 6 is about `class`.

Structs

A struct is a value type with a fixed layout known at compile time.

```
struct Counter:
    var count: Int
    var label: String

    def __init__(out self, label: String):
        self.count = 0
        self.label = label

    def increment(mut self):
        self.count += 1

    def describe(self) -> String:
        return self.label + ": " + String(self.count)
```

Fields are declared with `var`. Methods are ordinary `def`s whose first parameter is `self`, and the argument convention on `self` says what the method does:

First parameter	Meaning
`self`	Reads the value. The default.
`mut self`	Mutates it in place.

`var self`	Takes it by value; the method owns it.
`out self`	Initialises it. Used by `__init__`.
`deinit self`	Consumes it, ending its lifetime.

`Self` names the enclosing type, which matters in generic code and in trait requirements.

Static methods take no `self`:

```
    @staticmethod
    def zero() -> Self:
        return Counter("")
```

`@fieldwise_init`

When the initialiser would just assign each field in order, ask for it:

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64

var p = CGPoint(3.0, 4.0)
```

This is the shape almost every Cocoa geometry struct takes.

The lifecycle

Here Mojo diverges sharply from older writing about it, and from most other languages. There are four operations, and **three of them are spelled as `__init__` overloads**:

```
struct Buffer(Copyable, Movable, Deinitable):
    var data: Pointer[UInt8, MutUntrackedOrigin]
    var size: Int

    def __init__(out self, size: Int):              # construct
        ...

    def __init__(out self, *, copy: Self):          # copy
        ...

    def __init__(out self, *, deinit move: Self):   # move
        ...

    def __deinit__(deinit self):                    # destroy
        ...
```

If you have seen `__copyinit__` and `__moveinit__`, those names are gone. The keyword-only `copy:` and `deinit move:` arguments replace them, and there are zero uses of the old spellings left in the standard library.

Copying is explicit at the call site too:

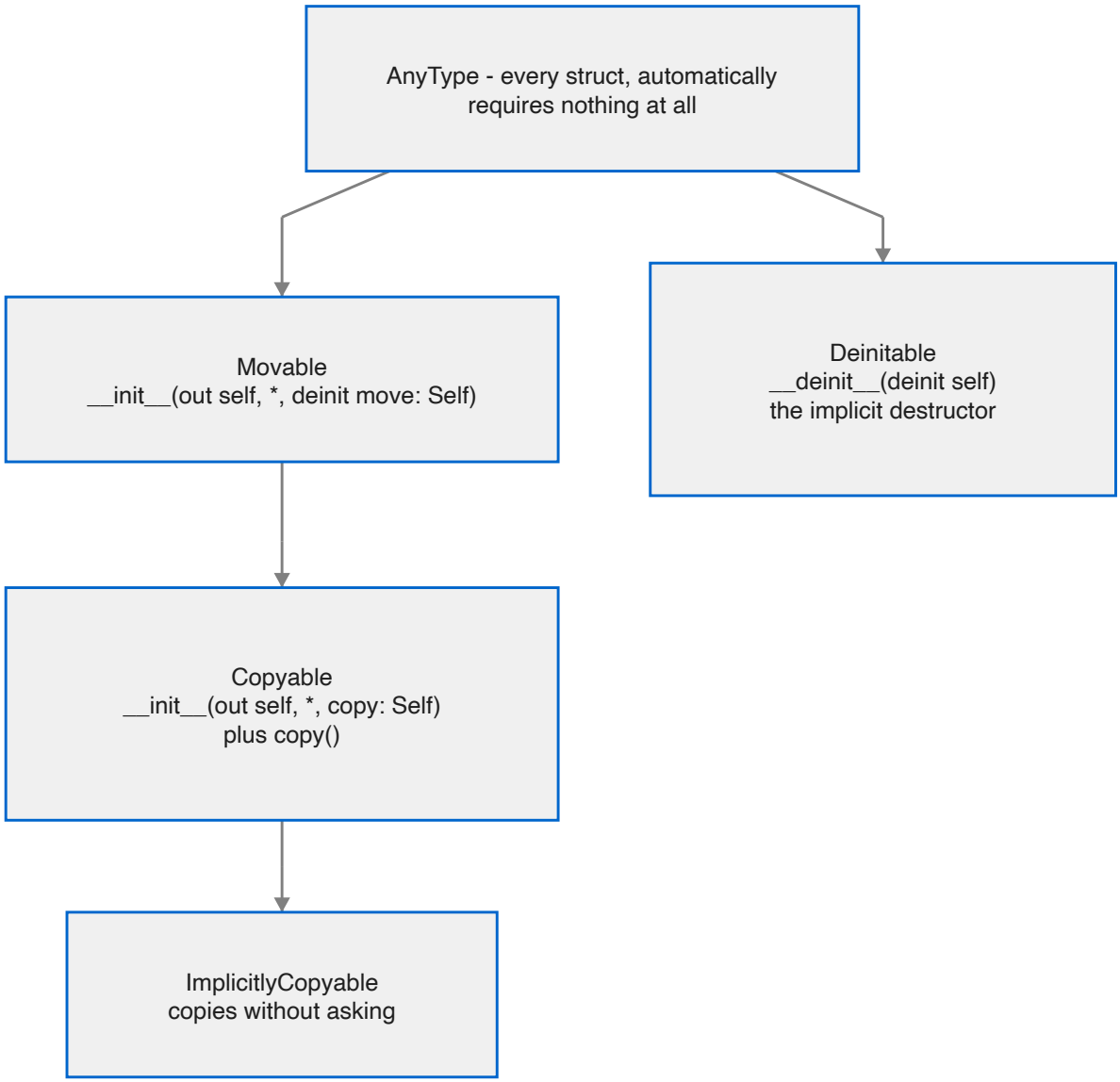
```
var b = a.copy()
```

`copy()` is a convenience for `Self(copy=self)` provided by the `Copyable` trait, and **overriding it is not allowed** — the trait says so directly. You customise the constructor, not the convenience.

Moving uses the transfer sigil:

```
var b = a^
```

The four traits everything rests on



AnyType is the floor. Every struct conforms to it automatically, and it requires nothing — *not even a destructor*. That is the unusual part, and the next section is about why.

Movable requires the move constructor. It also carries a compile-time flag, `__move_ctor_is_trivial`, that the compiler usually generates: true when the value can be moved by copying its bits with no side effects.

Copyable(Movable) requires the copy constructor and inherits movability. Copying is explicit by default.

ImplicitlyCopyable(Copyable) opts a type into being copied without an explicit `.copy()`.

Deinitable is the implicit destructor. Every type gets it by default.

Linear types, and why `AnyType` requires nothing

Most languages with strong lifetimes require every type to provide at least a trivial destructor, so the compiler can destroy a value whenever it decides the value is dead. Mojo's floor is lower than that on purpose.

A type may conform to `AnyType` but **not** to `Deinitable`. Such a type is called *linear*, and it has no implicitly-callable destructor at all. The compiler will not quietly clean it up; if you let one go out of scope without consuming it, that is a **compile error**.

```
struct MustBeCommitted(Deinitable where False):
    ...
    def commit(deinit self): ...
    def roll_back(deinit self): ...
```

The `Deinitable where False` conformance is how a type opts out. The effect is that the user must *choose* — commit or roll back — and cannot drift into neither by forgetting.

This is a real tool, not a curiosity. A linear type is a guard that some explicit action must happen later, enforced by the compiler rather than by review. You will not need it often; when you do, nothing else substitutes.

Traits

A trait declares requirements. A body of `...` means "conforming types must provide this".

```
trait Equatable:
    def __eq__(self, rhs: Self) -> Bool: ...
```

Three things traits can do that make them carry most of the weight classes would.

They inherit. `trait Comparable(Equatable)` requires everything `Equatable` does, plus its own.

They carry default implementations. A trait method with a real body is a default, and conforming types get it free:

```
trait Comparable(Equatable):
    def __lt__(self, rhs: Self) -> Bool: ...

    @always_inline
    def __gt__(self, rhs: Self) -> Bool:
```

```
        return rhs < self
```

Implement `__lt__` and `__eq__`; get `__gt__`, `__le__` and `__ge__` for nothing. This is where inherited behaviour lives in Mojo.

They carry associated types. A `comptime` member constrained by a trait:

```
trait IterableOwned:
    comptime IteratorOwnedType: Iterator

    def __iter__(var self) -> Self.IteratorOwnedType: ...
```

The conforming type chooses what `IteratorOwnedType` is, and refers to it as `Self.IteratorOwnedType`.

Composition

Traits combine with `&`, anywhere a trait is expected:

```
def once[T: Movable & Deinitable, //](...):
```

Conditional conformance

A generic type can conform to a trait *only when its parameter does*. This is the most expressive thing in the system, and `Optional` uses it heavily:

```
struct Optional[T: AnyType](
    Boolable,
    Copyable where conforms_to(T, Copyable),
    Deinitable where conforms_to(T, Deinitable),
    Equatable where conforms_to(T, Equatable),
    Movable where conforms_to(T, Movable),
    ...
):
```

`Optional[Int]` is copyable because `Int` is. `Optional[SomeLinearType]` is not, and the compiler knows without being told twice.

Generics

Struct and function parameters go in square brackets and are compile-time values:

```
struct Optional[T: AnyType]:
    ...

def largest[T: Comparable](a: T, b: T) -> T:
    return a if b < a else b
```

`Some[Trait]`

When a parameter exists only to name the argument's type, `Some` says so more briefly:

```
def foo[T: Intable, //](x: T) -> Int:   # the long way
def foo(x: Some[Intable]) -> Int:       # the same thing
```

The `//` marks parameters that are *inferred only* — never written at the call site.

Some[Trait] is not an existential. It is an alias that expands to an inferred generic parameter, so the call is still resolved and specialised at compile time. There is no boxing, no vtable and no dynamic dispatch hiding in it. `Some[Writer]` in a signature means "some specific type that writes, chosen at compile time", not "any writer, decided at run time".

That distinction is the whole difference between Mojo's object model and Objective-C's, and it is worth holding on to for the next chapter.

Operators

The dunder protocol will be familiar from Python. This compiler's standard library defines and dispatches:

Group	Methods
Arithmetic	`__add__ __sub__ __mul__ __truediv__ __floordiv__ __mod__ __pow__`
Unary	`__neg__ __pos__ __abs__ __invert__`
Bitwise	`__and__ __or__ __xor__ __lshift__ __rshift__`
Comparison	`__eq__ __ne__ __lt__ __le__ __gt__ __ge__`
Container	`__getitem__ __setitem__ __len__ __contains__ __iter__ __next__`
Conversion	`__int__ __float__ __bool__ __str__ __hash__`
Call and scope	`__call__ __enter__ __exit__`

`__enter__` and `__exit__` are what make `with autoreleasepool():` work — a context manager is just a struct with those two methods, which is exactly how `autoreleasepool` is built.

Text formatting goes through the `Writable` trait and a `write_to` method rather than only `__str__`.

What this means for Cocoa

Lay the two models side by side and the division of labour is obvious.

	Mojo	Objective-C
Keyword here	`struct`	`class`
Unit	value semantics	reference semantics

Layout	fixed at compile time	object header plus ivars, discoverable at run time
Inheritance	none	single, deep, pervasive
Dispatch	static, always	dynamic, always — `objc_msgSend`
Polymorphism	traits and generics, resolved at compile time	messages, resolved at run time
Lifetime	scope-based, compiler-enforced	reference counting
Adding a method later	impossible	routine — categories, runtime class building

Mojo has no runtime object system, and Cocoa is nothing *but* a runtime object system. So CocoaMojo does not try to make Mojo object-oriented. It does two narrower things:

It wraps Objective-C references in Mojo structs. `ObjCRef` is an ordinary struct with a `__deinit__` that calls `objc_release`. Cocoa's reference counting becomes Mojo's scope-based lifetime, with no new machinery on either side — the next chapter but one is entirely about this.

It builds real Objective-C classes. When Cocoa needs to send your code a message, a `class` declaration becomes a genuine registered ObjC class whose method implementations are Mojo. The dynamic dispatch is real; it just lands on static Mojo code.

Everything in the rest of this guide is one of those two moves. Knowing which one you are looking at makes the rest considerably easier to read.

4. Calling Cocoa

Every Objective-C method call is one C function call:

```
objc_msgSend(id self, SEL op, ...)
```

Binding that is not hard because of the call. It is hard because of everything the call depends on — does this class have this selector, what does it return, which register file does each argument travel in — and all of that lives in the SDK, where it is traditionally transcribed by hand, once, wrongly.

`msg_send` gets those facts from the database while your program compiles.

First: load the framework

Foundation arrives free — something in every process drags it in. **AppKit does not**, unless the binary was linked against it. In a JIT-run program (`mojo run`) nothing was, so `objc_getClass("NSApplication")` returns nil and every message to it silently no-ops. The app "runs", exits without a window, and produces no diagnostic anywhere.

That failure shape cost the fork real time. Call this first in anything windowed:

```
if not load_framework["AppKit"]():
    raise Error("could not load AppKit")
```

It is a `dlopen` with `RTLD_NOW`, idempotent and cheap after the first call.

Looking up a class

```
var cls = ObjCClass.lookup["NSString"]()
if cls.is_nil():
    print("NSString not registered — is Foundation linked?")
```

The name is a *parameter*, in square brackets, because it is resolved at compile time. `is_nil()` is worth checking once at startup: a nil class means the framework was never loaded, and every subsequent message will silently do nothing.

`ObjCClass` and `ObjCObject` are distinct types even though both are pointers at the C ABI, so you cannot pass one where the other is expected. Convert deliberately:

```
var as_obj = cls.as_object()
```

Sending a message

```
var n = msg_send[Int, "NSString", "length"](s)
```

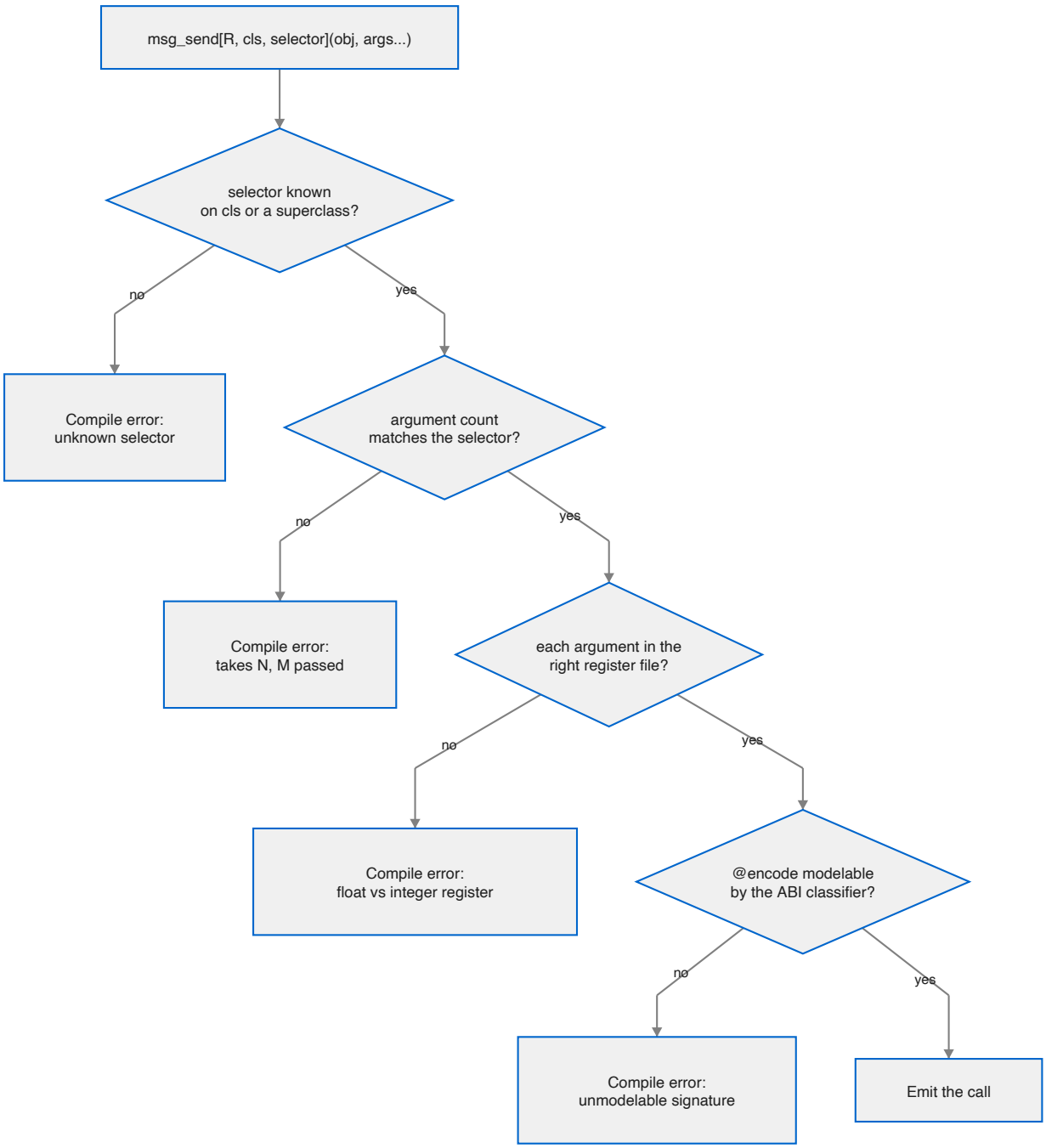
Read the parameters left to right: the **return type**, the **class the selector is looked up on**, and the **selector**. Then the receiver in parentheses.

For a class method, add `is_class=True`:

```
var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

Arguments follow the receiver. The colons in the selector tell you how many to expect — `stringWithUTF8String:` takes one, `length` takes none — and the compiler checks that you passed the right number.

What gets checked, and when



Four checks, all before code generation. Taking them in turn:

The selector must exist. The lookup walks the superclass chain in the metadata, so asking `NSMutableString` about `length` correctly finds `NSString`'s definition. A misspelling has nowhere to hide.

The argument count must match. This is the check that saves you from the worst class of bug. Call `characterAtIndex:` with no index and the callee reads whatever happened to be in the argument register — a plausible-looking number, a crash much later, or nothing at all. Here it is a compile error, and the fork ships it as a must-fail test.

Each argument must be in the right register file. On AAPCS64 a scalar float travels in `v0`–`v7` and everything else in the general-purpose registers. Passing an `Int` where the ABI wants a `Float64` is not a conversion; it is reading the wrong register. `msg_send` flags the cases it is certain about — a scalar float against an integer class, and the reverse — and stays quiet about structs and unknowns so there are no false positives.

The signature must be modelable. If the ABI classifier cannot make sense of a method's `@encode` string it answers `"?"`, and that is a compile error rather than a guess. This is the check that matters least often and matters most when it fires.

Return types

The return type parameter is what the C ABI sees, so pick the Mojo type that matches the Objective-C one:

Objective-C	Mojo return parameter
`id`, any object	`ObjCObject`
`NSUInteger`, `NSInteger`	`Int`
`BOOL`	`Bool`
`unichar`	`UInt16`
`double`, `CGFloat`	`Float64`
`const char *`	`OpaquePointer[MutUntrackedOrigin]`
`void`	assign to `_` and ignore

A `void` method still returns something as far as Mojo is concerned, so the idiom is:

```
_ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](app, delegate.ptr())
```

You will write `_ =` constantly. It is not a wart; it is the compiler refusing to let you silently discard a value.

Struct returns, and why arm64 makes them boring

On x86-64, a method returning a large struct must go through a completely different entry point, `objc_msgSend_stret`, with a hidden buffer pointer in `rdi` and the receiver shifted to `rsi`. Getting that wrong corrupts the stack.

On arm64 there is no `objc_msgSend_stret` and no `objc_msgSend_fpret`. They do not exist in this machine's libobjc, because AAPCS64 does not need them: a small aggregate comes back in `x0`–`x1`, a homogeneous float aggregate in `v0`–`v3`, and anything larger through the `x8` indirect-result register. All of that happens through the ordinary send.

So `cocoakb_msgsend_variant` always answers `"objc_msgSend"` here. The query is kept anyway, because its *other* job survives: an unmodelable signature still answers `"?"` and still fails the build.

A concrete case worth internalising: `-[NSValue rectValue]` returns a `CGRect`, which is 32 bytes. On x86-64 that is a `_stret` call. Here it classifies as `h4` — a homogeneous float aggregate of four doubles — and comes back in `v0`–`v3` from a plain `objc_msgSend`.

Protocol-typed receivers: `send`

`msg_send` needs a class name to look the selector up on. Sometimes you do not have one. Every Metal object is declared as a protocol — `id<MTLDevice>`, `id<MTLTexture>` — and so is every Cocoa delegate. The concrete class is a private implementation detail you cannot name.

For those, use `send`, which is keyed by selector alone:

```
var tex = send[ObjCObject, "newTextureWithDescriptor:"](device, desc.ptr())
```

The dispatch stub and the argument classes come from any class in the database that implements the selector, which is consistent across implementors. You keep the argument-count check and the register-file check. The only thing you give up is verification that this particular receiver responds to it.

The trade is explicit, and the error when a selector is implemented by nothing at all is clear:

```
std.objc: no class in the metadata implements selector 'newTextureWithDesc:',
so its dispatch ABI is unknown. Check the selector spelling.
```

Selectors

`sel[...]` registers a selector and caches it:

```
var s = sel["applicationDidFinishLaunching:"]()
```

The resolved `SEL` is cached in a per-selector global slot, deduplicated by name in the KGEN lowering, so every call site for a given selector shares one slot. After the first send, resolving a selector is a load and a branch-predicted-away null check — not a hash lookup and not a runtime registry call.

You rarely call `sel` directly; `msg_send` does it for you. You need it when talking to `class_addMethod` or `respondsToSelector:`.

Strings

`NSString` is the type you will convert most, and there are three levels of convenience.

The raw send, when you want to see the machinery:

```
var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

`nsstring`, for handing an autoreleased string to an AppKit setter that will retain it:

```
_ = msg_send[ObjCObject, "NSWindow", "setTitle:"](win, nsstring("Life").ptr())
```

And `NSString`, a leak-safe owning wrapper, when the string outlives the statement:

```
var greeting = NSString("Hello")
print(greeting.length())
print(greeting.to_string())
```

Going the other way, `ns_to_string` is the direction that used to be missing:

```
var text = ns_to_string(some_nsstring)
```

It copies out as UTF-8, so the result does not depend on the pool that owns the `NSString`. Underneath it is still `-UTF8String`, which you can call yourself when you want the raw pointer:

```
var p = msg_send[
    OpaquePointer[MutUntrackedOrigin], "NSString", "UTF8String"
](s)
var text = String(unsafe_from_utf8_ptr=p.bitcast[c_char]())
```

Errors: `NSError` becomes `raises`

Cocoa's error convention predates the exceptions it never adopted. A fallible method takes a trailing `error:` out-parameter (`NSError **`) and signals failure **through its return value** — nil for object returns, `NO` for `BOOL` returns. The error object is only meaningful when the return value says failure.

Checking the out-parameter instead of the return value is the classic Cocoa bug. Ignoring it throws the diagnosis away — which is exactly what this codebase used to do, with an `err_out()` sink that discarded every message.

Two wrappers convert the convention to Mojo's:

```
# Object convention: nil means failure.
var contents = msg_send_raising[
    "NSString", "stringWithContentsOfFile:encoding:error:", is_class=True
](ns_string_cls, path.ptr(), Int(4))

# BOOL convention: NO means failure. Returns nothing.
msg_send_raising_check[
    "NSString", "writeToFile:atomically:encoding:error:"
](nsstring("round trip"), path.ptr(), Bool(True), Int(4))
```

Do not pass the error argument. The wrapper creates the stack slot and appends it for you. Pass the message arguments without it.

On failure you get a Mojo `Error` whose message carries Cocoa's own diagnosis — the `localizedDescription`, plus domain and code:

```
stringWithContentsOfFile:encoding:error:: The file "really-not-here.txt"
couldn't be opened because there is no such file. (NSCocoaErrorDomain 260)
```

Everything `msg_send` checks is still checked. The selector must exist, the register-file classes are verified, and the argument count includes the error slot the wrapper supplies — all at compile time.

Two limits worth knowing. The wrappers are declared at explicit arities up to five leading arguments, because Mojo permits nothing after an unpacked `*args` so the slot cannot be appended to a forwarded pack. That covers the SDK's convention comfortably, and a new arity is four lines. And if an API returns failure without writing an error, you get a message saying so rather than a crash.

Enum constants

Cocoa's named constants come from BridgeSupport, resolved at compile time:

```
comptime titled = cocoakb_enum_value["NSWindowStyleMaskTitled"]()
comptime utf8 = cocoakb_enum_value["NSUTF8StringEncoding"]()   # 4
```

Use these instead of typing the numbers. The one place you will be tempted to cheat is style masks, where `15` is quicker to write than four lookups — the example applications do exactly that, and it is the kind of shortcut that survives right up until Apple renumbers something.

Extern object constants

Constants like `NSForegroundColorAttributeName` are not values the metadata can hand over at compile time. They are globals whose address the linker resolves. `extern_object` takes a link-time reference to the data symbol and loads the pointer out of it:

```
var key = extern_object["NSForegroundColorAttributeName"]()
```

Struct layouts

Declare the struct in Mojo, then assert that your declaration agrees with the SDK:

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64

@fieldwise_init
struct CGSize(Copyable, Movable):
    var width: Float64
    var height: Float64

@fieldwise_init
struct CGRect(Copyable, Movable):
    var origin: CGPoint
    var size: CGSize

comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()
comptime assert align_of[CGRect]() == cocoakb_struct_align["CGRect"]()
```

```
comptime assert cocoakb_field_offset["CGRect", "size"]() == 16
```

This is the part people skip, and it is the part that pays. A wrong field offset does not fail loudly — it reads a filename out of the middle of a timestamp. Most sample code you will find online quotes 32-bit offsets; `WIN32_FIND_DATAW`'s equivalent trap exists on every platform. Assert, and the build tells you.

Once the assertions pass you can pass the struct by value straight through a send, and the C ABI does the register allocation:

```
win = msg_send[
    ObjCObject, "NSWindow", "initWithContentRect:styleMask:backing:defer:"
](
    win,
    CGRect(CGPoint(100.0, 100.0), CGSize(1080.0, 720.0)),
    Int(15),
    Int(2),
    Bool(False),
)
```

5. Ownership and memory

Cocoa manages memory by reference counting. An object is created at +1, every `retain` adds one, every `release` takes one away, and at zero it is deallocated. Autorelease pools defer a release to the end of a scope.

Mojo has value semantics and deterministic destruction. CocoaMojo binds the two so that the Objective-C reference count follows the lifetime of the Mojo value holding it — which means **you opt out of correct memory management, never into it**.

`ObjCRef` owns a +1

```
struct ObjCRef(Movable):
    ...
```

An `ObjCRef` holds one owned reference. It releases when the Mojo value is destroyed. There are two ways to make one, and picking the right one is the whole skill.

```
var owned = ObjCRef(adopt=obj)    # you already own a +1
var shared = ObjCRef(retain=obj)  # you only borrowed it; add a +1
```

The rule that decides which is the oldest rule in Cocoa, and it is mechanical:

Selector you called	You own the result?	Use
`alloc`, `new`, `copy`, `mutableCopy`	yes, +1	`adopt=`
anything else	no, borrowed	`retain=`

Get this backwards in the `adopt` direction and you over-release, which crashes somewhere unrelated. Get it backwards in the `retain` direction and you leak.

The classic +1 chain

```
def make_array() -> ObjCRef:
    var cls = ObjCClass.lookup["NSMutableArray"]()
    var allocated = msg_send[
        ObjCObject, "NSMutableArray", "alloc", is_class=True
    ](cls.as_object())
    var inited = msg_send[ObjCObject, "NSObject", "init"](allocated)
    return ObjCRef(adopt=inited)
```

`alloc` returns +1, `init` consumes and returns it, and `adopt=` takes over. The caller cannot forget to release, because there is nothing to forget: the `ObjCRef` releases when it dies.

The fork's ownership test cycles a million of these and watches memory stay flat.

Copying is an explicit retain

`ObjCRef` is `Movable` but not implicitly `Copyable`. Sharing an Objective-C object means retaining it, and the design makes that visible:

```
var a = make_array()
var b = a.copy()     # a visible +1
```

If you want to move rather than share, use the transfer sigil:

```
var b = a^
```

Autorelease pools

```
with autoreleasepool():
    var s = make_autoreleased()
    print("live inside the pool:", not s.is_nil())
# pool drained here
```

`autoreleasepool` pushes a pool on entry and pops it on exit. It guards against a double pop internally, because popping a token twice corrupts the pool page and produces a hard crash a long way from the cause.

Wrap `main` in one. Wrap any loop that creates many temporary Cocoa objects in one. Do **not** wrap an AppKit callback in one — AppKit's own event dispatch already has a pool active, and every object you read from an event is autoreleased by the caller.

Returning an object without leaking

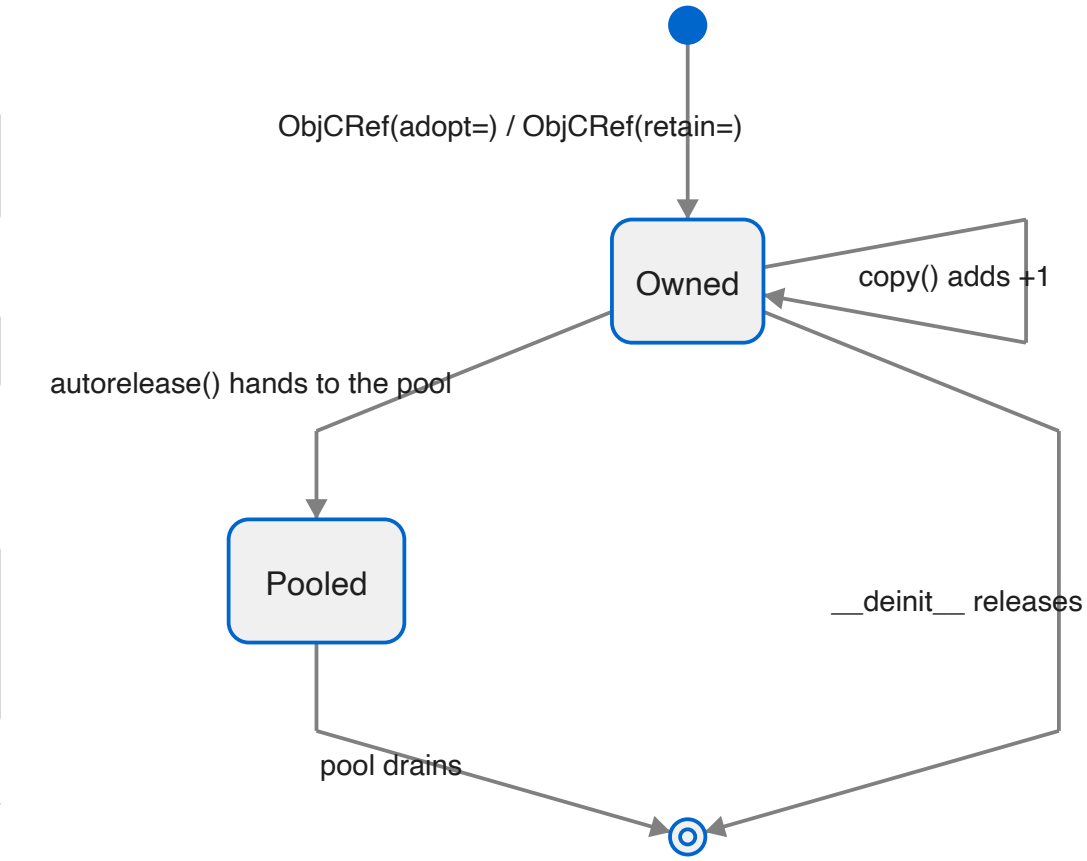
Sometimes you want to hand an object back without making the caller own it — exactly what an Objective-C method returning an autoreleased object does. `autorelease()` consumes the `ObjCRef` and gives the object to the current pool:

```
def make_autoreleased() -> ObjCObject:
    var owned = make_array()
    return owned^.autorelease()
```

Note the `^`. `autorelease` takes `deinit self`, so the reference is consumed; without the sigil you get a compile error.

The returned `ObjCObject` is valid until the pool drains. Using it afterwards is a use-after-free, and no amount of compile-time checking can save you from that one — it is the one place in CocoaMojo where the discipline is yours.

The lifetime, as a state machine



`let` for the bindings you never rebind

Everything above works the same whether you bind with `var` or `let`. The ARC behaviour lives in `ObjCRef`, not in the keyword.

```
let obj = make_array()          # +1 at bind, released at scope exit
let s = nsstring("hello")
```

What `let` adds is that the *binding* cannot be reassigned, which is worth having precisely because Cocoa code is full of handles you obtain once and then only send messages to. If you never rebind it, say so; the compiler will hold you to it.

The object stays mutable. `let win = ...` then `setTitle:` is fine — you are mutating the object, not rebinding the name.

Weak references

`ObjCRef` is the strong half of the story. `ObjCWeakRef` is the other half, and you need it more often than you might expect, because **Cocoa's delegate convention is weak**. A window does not own its delegate; an observer does not own its target. Holding an `ObjCRef` where Cocoa expects weakness is a retain cycle that leaks both sides.

```
var strong = make_object()
var weak = ObjCWeakRef(strong.object())

var loaded = weak.load()        # an ObjCRef: the object at +1, or nil
if loaded.is_nil():
    print("gone")
```

It is a **zeroing** weak reference: while the object lives, `load()` returns it; after the last strong reference goes, `load()` returns nil. A weak reference to nil is legal and simply loads nil.

Three details that matter in use.

load() returns an owned ObjCRef, not a bare handle. It goes through `objc_loadWeakRetained`, so the object cannot be deallocated between your nil-check and your use. That is the entire reason to load a weak reference rather than peek at it.

Copying is explicit, matching `ObjCRef`:

```
var weak2 = weak.copy()         # an independent registration
```

There is one small heap allocation per weak reference, and the reason is worth understanding. The Objective-C runtime tracks a weak reference *by the address of the slot holding it*. Mojo values move. If the slot were a struct field, any move would leave the runtime pointing into a dead stack frame — a corruption with no diagnostic. So the slot lives on the heap, its address stays stable for the life of the value, and moving the struct moves only a pointer. Delegates are few; the allocation is not a problem.

A rough edge

Assigning into an uninitialised `var` of a type with a `__deinit__` destroys garbage first and crashes. This is pre-existing and not specific to weak references, but it bites here because the natural shape —

```
var weak: ObjCWeakRef          # then assign later
```

— is exactly that pattern. Bind it in a helper scope and return it instead.

Buffers that Cocoa callbacks must reach

This one is not about Objective-C at all, and it will catch you.

Mojo destroys a value at its **last use**, not at the end of scope. So a `List` whose `.unsafe_ptr()` you stash in a global is freed immediately after that line, and every stored pointer dangles. The symptom arrives much later as a corrupted allocator, nowhere near the cause.

When memory has to outlive Mojo's view of it, own it outside Mojo:

```
def alloc_zeroed(count: Int, size: Int) -> Int:
    var sym = _sym["calloc"]()
    var call = Pointer(to=sym).unsafe_bitcast[
        def(Int, Int, /) thin abi("C") -> P
    ]()[]
    return Int(call(count, size))
```

Explicit, unowned, and correct. The alternative — hoping a `List` lives long enough — is not a smaller amount of unsafety, only a less visible one.

Where callback state lives

Cocoa callbacks are bare C function pointers. They get no closure and no context argument you did not arrange yourself. State they need must therefore live somewhere reachable by name.

```
comptime g_running = named_global["life.running", Int]
comptime g_gen = named_global["life.gen", Int]

# read and write through the pointer
g_running()[] = 1
var generation = g_gen()[]
```

`named_global` gives you one zero-initialised process global per name, shared across every call site, because KGEN deduplicates the global by name. It is the plainest possible answer to a real constraint, and it is better than the alternative of smuggling a pointer through `setRepresentedObject:` and casting it back.

A short checklist

Use `adopt=` after `alloc`, `new`, `copy`, `mutableCopy`; `retain=` otherwise. Use `^` when consuming an `ObjCRef`. Wrap `main` and long loops in `autoreleasepool`, but not AppKit callbacks. Allocate anything a callback must reach outside Mojo's ownership, and reach it through `named_global`.

6. Letting Cocoa call you

Everything interactive in Cocoa works the same way: you supply an object, and the framework sends it a selector. Target/action, delegates, notification observers, timers — all of it.

To supply such an object you need an Objective-C class. In cocoa-mojo you declare one with `class`, and the compiler does the rest.

`class` declares an Objective-C class

The whole of it is a declaration:

```
class ExampleActions:
    def buttonClicked_(self, sender: ObjCObject):
        clicks()[] += 1
```

That is a real Objective-C class. The compiler registers it with the runtime, derives the selector, works out the type encoding, builds the trampoline, and gives you an instance:

```
let actions = ObjCObject(ExampleActions().__objc_id)
```

No `ObjCClassBuilder`, no encoding string, no `IMP`, and no `cmd: P` slot.

Method names become selectors

A trailing underscore is a colon.

Mojo method	Selector
`acceptsFirstResponder`	`acceptsFirstResponder`
`buttonClicked_`	`buttonClicked:`
`mouseDown_`	`mouseDown:`
`applicationShouldTerminateAfterLastWindowClosed_`	`applicationShouldTerminateAfterLastWindow`

The number of underscores must equal the number of arguments after `self`, and a mismatch is a compile error naming both counts.

That diagnostic earns its place in the *other* direction. Writing `drawRect` where you meant `drawRect_` derives a nullary selector, registers perfectly, and then never receives a single draw — the framework keeps sending `drawRect:` to a class that answers `drawRect`. Nothing crashes, nothing appears. It is now a compile error.

A leading underscore means the method is Mojo's own and never reaches the runtime. Without that rule every snake_case helper in a class would derive a nonsense selector like `my:helper` and then be rejected for a colon it never wanted — which would make a Cocoa class a hostile place to put a private method. `_tab_width` is simply private, exactly as Mojo and Python already read a leading underscore, and the dunders come along for free.

To spell a selector that the underscore rule cannot produce, say so:

```
    @objc("insertText:replacementRange:")
    def insert_text(self, text: ObjCObject, range: NSRange): ...
```

Superclass and protocols

```
class LifeView(NSView):          # subclass an SDK class
class LifeDelegate:              # no base means NSObject
class Editor(NSView, NSTextInputClient):   # first base is the superclass,
                                           # the rest are protocols
```

Protocols are adopted through `class_addProtocol` — real conformance, not merely having the methods. AppKit does ask: `conformsToProtocol:` reportedly cost a day on `NSTextInputClient` before that was understood.

An unknown superclass is caught at compile time, so a typo in a class name fails the build rather than producing a class that inherits from nothing.

Encodings are looked up, then derived

If the selector already exists on the superclass chain, the encoding comes from the SDK database, and a signature that disagrees with it is a compile error **quoting the database's encoding**. If the SDK has never heard of the selector — which is the normal case for target/action and delegate methods you invent — the encoding is derived from your Mojo signature.

Either way you do not write `"v@:@"` again.

Methods may raise

This is the part that changes how the code reads:

```
class ExampleActions:
    def buttonClicked_(self, sender: ObjCObject):
        with autoreleasepool():
            _ = msg_send[ObjCObject, "NSTextField", "setStringValue:"](...)
```

Closed:

No `try`, and no `fn`. A `class` method body is a `def` and **may raise**; the synthesized trampoline catches at the Objective-C boundary, because unwinding into `objc_msgSend` is undefined behaviour. On a raise the boundary reports — `NSLog` and continue, by default — and returns a zero value.

You can still write `fn` methods where you want the strict contract and no catch machinery. But the reason the examples read cleanly is that the common case no longer needs one.

What a class does not have yet

Fields. `var` members on a class are designed — one hidden ivar holding a boxed Mojo struct, with synthesized `init` and `dealloc` — but not implemented. Until they are, callback state lives where it

always has:

```
comptime clicks = named_global["example.clicks", Int]
comptime label_addr = named_global["example.label", Int]
```

`examples/life/main.mojo` still carries fourteen of these for exactly this reason. When fields land, most of them become ordinary members with real construction and destruction — which is the thing `named_global` can never give you.

Also not in this version: nested classes, class-level `comptime` parameters, and Mojo-trait conformances. A class is not a struct.

Instances

`ClassName()` registers the class if it is not registered yet, then allocs and inits. Registration is idempotent — a second instance costs one `objc_getClass`.

```
var probe = Probe()
var obj = ObjCObject(probe.__objc_id)
```

`__objc_id` is the raw `id`. A class reference is register-passable, retains on copy and releases on destruction — which is what an Objective-C reference *is*, and the same shape `ObjCRef` already had.

Sending to your own selectors

One asymmetry to know. `msg_send` checks the selector against the SDK, so it cannot send a selector you invented:

```
# buttonClicked: is ours — msg_send will not have it
var responds = msg_send[Bool, "NSObject", "respondsToSelector:"](
    obj, sel_dynamic("buttonClicked:")
)
```

`sel_dynamic` registers a selector by name at run time, and `respondsToSelector:` is what a button asks before it sends anything anyway.

The older way: `ObjCClassBuilder`

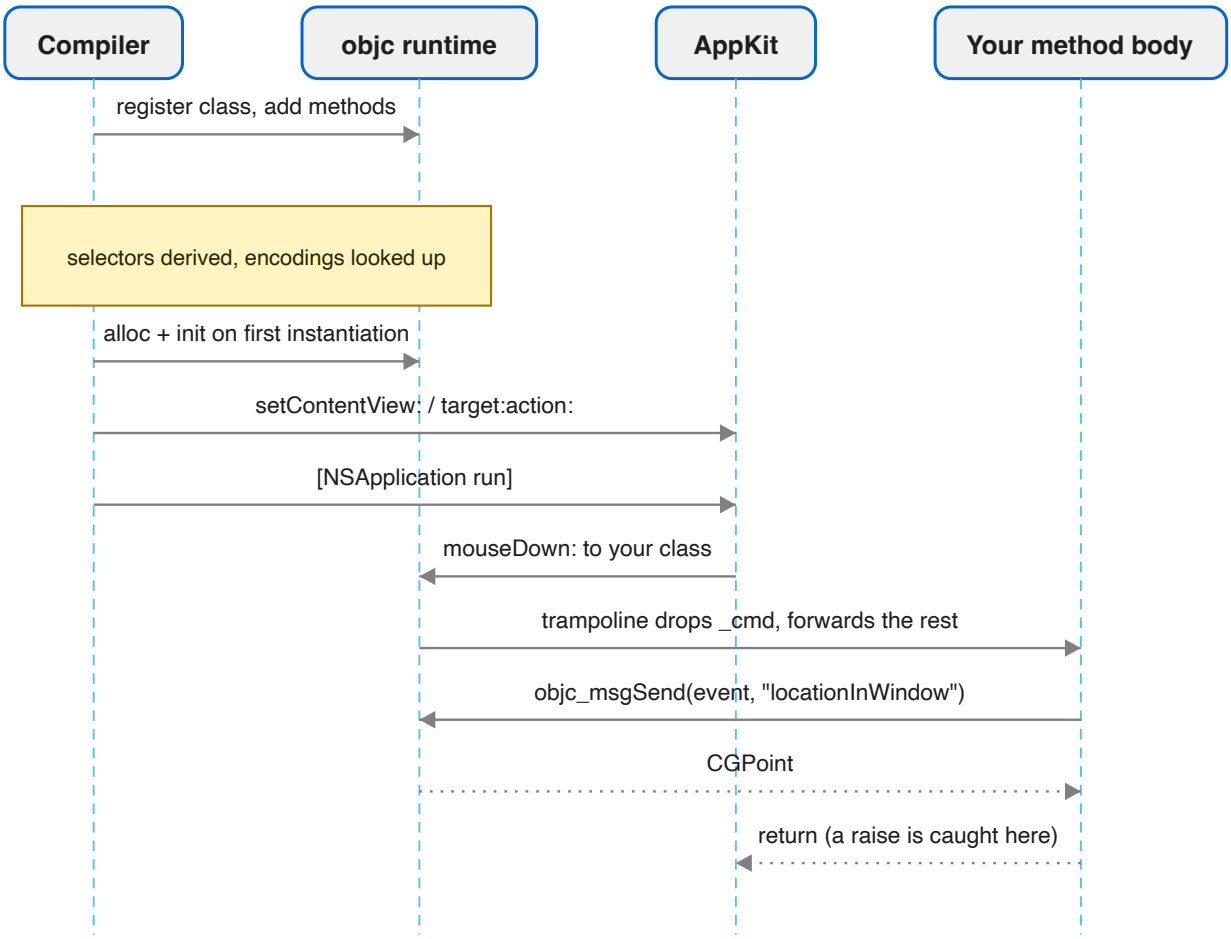
Everything above replaces a mechanism that still exists and still works. You will meet it in `spikes/`, which predate `class`:

```
var vb = ObjCClassBuilder["NSView"]("LifeView")
vb.add_method["mouseDown:"](on_mouse_down)
vb.add_method["tick:", encoding="v@:@"](on_tick)
var LifeView = vb^.register()
```

with callbacks written as `fn`s carrying the `self_: P, cmd: P` prefix by hand. It is the layer `class` is built on, and it remains the escape hatch for a method shape the class syntax does not cover. For new

code, declare a class.

The round trip



The trampoline is the piece you never see. It carries the `(self, _cmd, args…)` shape the runtime requires, drops the `_cmd` slot, forwards the rest to your method, and catches anything the body raises so it cannot unwind into `objc_msgSend`.

Verifying it worked

`respondsToSelector:` is the cheap check, and worth doing once during bring-up:

```
var responds = msg_send[Bool, "NSObject", "respondsToSelector:"](inst, s)
print("responds:", responds)
```

If that prints `False`, the usual causes are a selector spelled differently from the one you added, or a `register()` that never ran.

Delegates

A delegate is an object implementing the right selectors — so it is a class with those methods on it:

```
class LifeDelegate:
    def applicationShouldTerminateAfterLastWindowClosed_(
        self, app: ObjCObject
    ) -> Bool:
        return True

# ...
let delegate = ObjCObject(LifeDelegate().__objc_id)
_ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](app, delegate.ptr())
```

Because the selector is a real AppKit one, its encoding comes from the database and you never type `"c@:@"` — which, incidentally, is the encoding people most often get wrong, since `BOOL` is `c` and not `B`. Get the signature wrong and you get a compile error quoting what the SDK expects.

Where a delegate must *declare* conformance rather than merely have the methods, name the protocol as a base:

```
class Editor(NSView, NSTextInputClient):
    ...
```

Reading arguments out of an event

Callback arguments arrive as raw pointers. Wrap and send:

```
def event_has_shift(event: ObjCObject) -> Bool:
    var flags = msg_send[Int, "NSEvent", "modifierFlags"](event)
    return (flags & 131072) != 0    # NSEventModifierFlagShift
```

Arguments arrive typed now — a `class` method declares `event: ObjCObject` rather than a raw `P` — so most of the wrapping the old callbacks needed is gone.

That magic number should be `cocoakb_enum_value["NSEventModifierFlagShift"]()`. The example applications hardcode several of these, and it is the one habit in them worth not copying.

Reading a keystroke is the same pattern with a string on the end:

```
class LifeView(NSView):
  def keyDown_(self, event: ObjCObject):
    var chars = msg_send[
        ObjCObject, "NSEvent", "charactersIgnoringModifiers"
    ](ObjCObject(Int(event)))
    if chars.is_nil():
        return
    var p = msg_send[P, "NSString", "UTF8String"](chars)
    if Int(p) == 0:
        return
    var s = String(unsafe_from_utf8_ptr=p.unsafe_bitcast[c_char]())
    if len(s.as_bytes()) == 0:
```

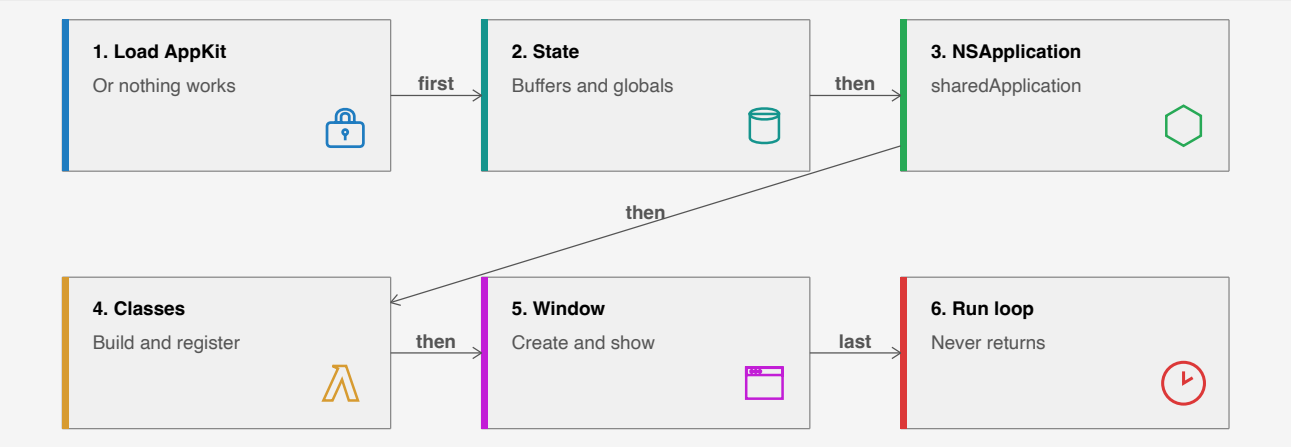
```
        return
    var c = s.as_bytes()[0]
    if c == UInt8(ord(" ")):
        toggle_running()
```

Three nil checks before touching anything. AppKit will hand you a nil string for a dead key, and a null `UTF8String` for an empty one.

7. A complete application

Everything so far assembles into one shape. This chapter is that shape, in the order it has to happen, with the reasons.

The skeleton



Order matters. A class must be registered before anything is instantiated from it, and the application object must exist before a window is made key. Getting either backwards produces a silent failure rather than an error.

Step 0 — load AppKit

```
def main() raises:
    if not load_framework["AppKit"]():
        raise Error("could not load AppKit")
```

Nothing linked AppKit into a `mojo run` process, so without this every AppKit class lookup returns nil and every message to it silently does nothing. The program starts and exits with no window and no diagnostic. Put it first.

Step 1 — state, before anything else

```
    g_alive()[] = alloc_zeroed(CELLS, 1)
    g_next()[] = alloc_zeroed(CELLS, 1)
    g_frame()[] = alloc_zeroed(PIXELS, 4)
    g_running()[] = 1
    randomize()
```

Do this first because the callbacks you are about to install can fire the moment the run loop starts, and a callback that reads a null global crashes on the first frame.

Step 2 — the application object

```
    with autoreleasepool():
        var NSApplication = ObjCClass.lookup["NSApplication"]()
        var app = msg_send[
            ObjCObject, "NSApplication", "sharedApplication", is_class=True
        ](NSApplication.as_object())
        _ = msg_send[Bool, "NSApplication", "setActivationPolicy:"](app, Int(0))
```

`setActivationPolicy:` with `0` is `NSApplicationActivationPolicyRegular` — the difference between a real application with a Dock icon and menu bar, and a process that puts a window on screen that cannot be focused. Skipping it is the most common reason a first CocoaMojo window appears dead.

Step 3 — classes

The delegate:

```
        var db = ObjCClassBuilder("LifeDelegate")
        db.add_method["applicationShouldTerminateAfterLastWindowClosed:"](
            should_terminate
        )
        var delegate = new_instance(db^.register())
        _ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](
            app, delegate.ptr()
        )
```

The view, whose event handlers are Mojo functions:

```
        var vb = ObjCClassBuilder["NSView"]("LifeView")
        vb.add_method["mouseDown:"](on_mouse_down)
        vb.add_method["mouseDragged:"](on_mouse_dragged)
        vb.add_method["keyDown:"](on_key_down)
        vb.add_method["acceptsFirstResponder"](accepts_first_responder)
        var LifeView = vb^.register()
```

`acceptsFirstResponder` returning `True` is what makes the view eligible to receive key events at all. Without it `keyDown:` never fires and the view looks broken in a way that gives no clue.

Step 4 — window and view

```
        var win = msg_send[ObjCObject, "NSWindow", "alloc", is_class=True](
            ObjCClass.lookup["NSWindow"]().as_object()
        )
        win = msg_send[
            ObjCObject,
            "NSWindow",
            "initWithContentRect:styleMask:backing:defer:",
        ](
```

```
            win,
            CGRect(CGPoint(100.0, 100.0), CGSize(1080.0, 720.0)),
            Int(15),
            Int(2),
            Bool(False),
        )
        g_window()[] = win.addr()
```

The `15` is a style mask — titled, closable, miniaturisable, resizable — and should really be four `cocoakb_enum_value` lookups combined. The `2` is `NSBackingStoreBuffered`.

Then the view, and making the window visible:

```
        var view = msg_send[ObjCObject, "NSView", "alloc", is_class=True](
            LifeView.as_object()
        )
        view = msg_send[ObjCObject, "NSView", "initWithFrame:"](view, frame)
        _ = msg_send[ObjCObject, "NSWindow", "setContentView:"](win, view.ptr())
        _ = msg_send[ObjCObject, "NSWindow", "makeKeyAndOrderFront:"](
            win, ObjCObject(0).ptr()
        )
        _ = msg_send[
            ObjCObject, "NSApplication", "activateIgnoringOtherApps:"
        ](app, Bool(True))
```

Step 5 — the run loop

```
    var app2 = msg_send[
        ObjCObject, "NSApplication", "sharedApplication", is_class=True
    ](ObjCClass.lookup["NSApplication"]().as_object())
    _ = msg_send[ObjCObject, "NSApplication", "run"](app2)
```

`run` does not return until the application terminates. Note that it is deliberately **outside** the `autoreleasepool` block: the pool should drain the setup objects before the loop begins, and the loop maintains its own pools per event cycle.

Re-fetching `sharedApplication` rather than reusing `app` is not superstition — `app` was bound inside the `with` block that has now ended.

What you get

The fork ships three applications built exactly this way.

`spikes/life/life.mojo` is Conway's Life with mouse drawing, pause and single step, and cells coloured by age so you can see a pattern's structure rather than a flat mask. Six hundred lines, and the only non-Cocoa part is the simulation.

`spikes/playground/playground.mojo` is a Mojo editor and runner written in Mojo — a thousand lines, a split view, syntax highlighting, and a child process.

`spikes/mandelbrot/mandelbrot.mojo` draws through a `CAMetalLayer`. It is the one example that depends on the GPU stack, which on Apple Silicon is not finished; treat it as a reading example rather

than a running one.

Debugging, honestly

There is no source-level debugging here yet. What you have is `print`, `respondsToSelector:` during bring-up, and the compile-time checks doing more work than they would in most languages.

The three failure modes worth recognising:

A window appears but ignores the keyboard: `acceptsFirstResponder` is missing or returns `False`.

A window appears but cannot be focused and has no Dock icon: `setActivationPolicy:` was not called.

A crash inside `objc_msgSend` with a plausible-looking receiver: an object was released while Cocoa still held it. Look for an `ObjCRef` that went out of scope, or a `new_instance` result that was never retained.

The first two are configuration. The third is the one the compile-time checks cannot reach, which is why chapter 5 spends so long on it.

8. Concurrency and blocks

Grand Central Dispatch is the concurrency story for a Cocoa app: UI work belongs to the main queue and everything else hops between queues. It is a C API in libSystem, so there is nothing to link.

The reason it lands cleanly here is a coincidence of contracts. Every GCD operation exists in two forms — a block form, and an `_f` form taking a bare C function pointer plus a context word — and **the _f form is exactly the revived fn**: thin, non-raising, C ABI. So most of `std.objc.dispatch` is direct calls with no adaptation layer at all.

```
from std.objc.dispatch import (
    Semaphore, async_f, global_queue, main_queue, sync_f, with_block,
)
```

Queues

```
var q = global_queue()      # the default-priority concurrent queue
var m = main_queue()        # where UI work belongs
```

`main_queue()` is worth a note: the main queue is not a function in C but a global, `_dispatch_main_q`, and the `dispatch_get_main_queue()` macro takes its address. The library references it the same way it references the `msg_send` stubs — as a link-time symbol.

Dispatching an `fn`

```
fn set_flag(ctx: P) -> None:
    named_global["app.flag", Int]()[] = 42

sync_f(q, ctx, set_flag)     # runs before returning
async_f(q, ctx, set_flag)    # returns immediately
```

The context word is how you get data to the work function, because an `fn` has no closure. In practice you will often reach for `named_global` instead, the same way Cocoa callbacks do — see Where callback state lives.

`Pointer` is non-nullable, so when the work function ignores its context you still have to hand it a real address rather than null. Any stable one will do.

Waiting: `Semaphore`

```
var sem = Semaphore()
var ctx = P(unsafe_from_address=sem._sem)

async_f(q, ctx, add_and_signal)
async_f(q, ctx, add_and_signal)
sem.wait()
sem.wait()
```

One `wait()` per expected signal. This is the rendezvous the dispatch spike uses to prove three concurrent `async_f` calls all ran.

Blocks

Plenty of modern Cocoa is block-only, and blocks have their own ABI — isa, flags, invoke pointer, descriptor, copy and dispose helpers. That sounds like a lot of machinery to build.

It is not, because of the correspondence the design turns on: **a thin fn is exactly a global block.** No captures means `_NSConcreteGlobalBlock` and no copy/dispose helpers at all, so the library can build the 32-byte block literal around any `fn` on the stack.

```
fn block_work() -> None:
    var n = named_global["app.count", Int]()
    n[] = n[] + 1

with_block[block_work](q, wait=True)     # sync: no copy needed
with_block[block_work](q, wait=False)    # async: the runtime Block_copy's it
```

The `wait=True` path never escapes the frame, so the stack literal is enough. The `wait=False` path does escape, and dispatch's own `Block_copy` memmoves the 32 bytes off your frame — safe precisely because there are no captures to deep-copy and the descriptor is immortal.

What is not here yet

Capturing closures as heap blocks. That needs synthesized copy and dispose helpers riding the existing closure emitter, and it is explicitly a later phase. If an API demands a block that captures, you are outside what the library covers today.

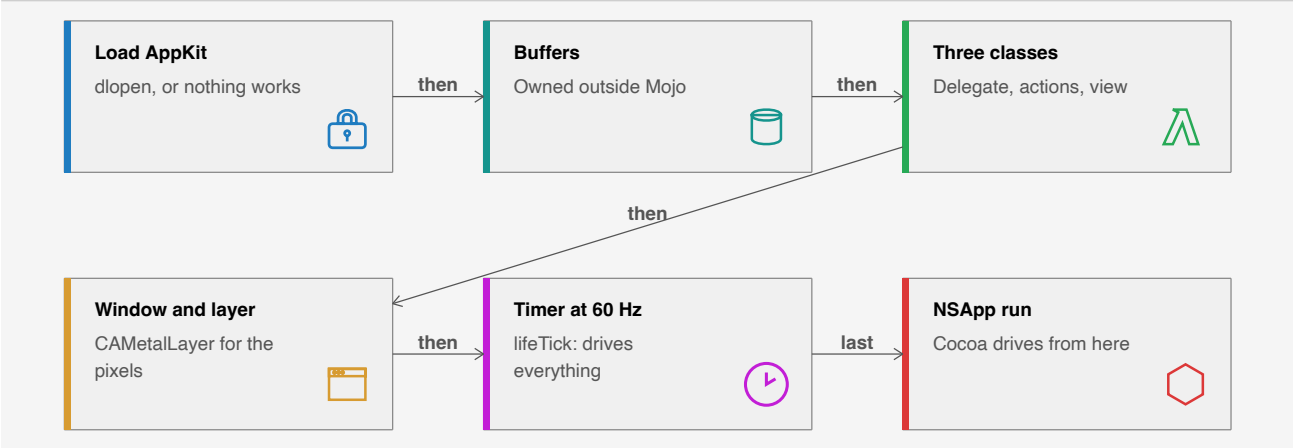
The practical consequence is the one you have already met everywhere else in this guide: state that a callback needs must live somewhere reachable without a closure. `named_global` is the answer here too.

9. A demo, walked through

This chapter reads one complete program: `examples/life/main.mojo` in the fork. Conway's Life, 644 lines, in a real window — mouse drawing, keyboard control, a 60 Hz timer, and cells coloured by age so you can see the structure of a pattern rather than a flat mask. Rendering goes through a `CAMetalLayer`.

It is worth reading because it is the current shape of a CocoaMojo application: three `class` declarations, no `ObjCClassBuilder`, no encoding strings, no `cmd` slots.

Everything below is quoted from the source.



The three classes

That is the part worth studying, and it is short.

A delegate

```
class LifeDelegate:
    def applicationShouldTerminateAfterLastWindowClosed_(
        self, app: ObjCObject
    ) -> Bool:
        return True
```

No base, so it subclasses `NSObject`. One method, whose trailing underscore makes the selector `applicationShouldTerminateAfterLastWindowClosed:`. AppKit declares that selector, so the encoding is looked up rather than derived, and a wrong signature would be a compile error quoting the SDK.

Returning `True` is what lets `[NSApp run]` ever finish: closing the window ends the app.

A timer target, and a lesson in its docstring

```
class LifeActions:
    """The timer's target.
```

```
    Named `lifeTick:` rather than `tick:` on the compiler's advice: the SDK
    declares a `tick:` on CASecureFlipBookLayer taking a double, and a
    selector we invent that collides with one the SDK knows would have been
    registered with that shape -- `v@:d` where we meant `v@:@`. The
    diagnostic said so."""

    def lifeTick_(self, timer: ObjCObject):
        advance_tick()
```

This is the single best illustration in the tree of what the compile-time checking buys.

The obvious name for a timer callback is `tick:`. But `tick:` already exists in the SDK — on `CASecureFlipBookLayer`, taking a `double` — and because encodings are *looked up* when the selector is known, the class would have registered `v@:d` while the code meant `v@:@`. The timer would then have handed a double where an object pointer was expected.

That is a silent, hard-to-find corruption in the old world. Here it was a diagnostic, and the fix was to pick a name nobody else declares — at which point the encoding is derived from the Mojo signature, correctly.

A view subclass

```
class LifeView(NSView):
    def mouseDown_(self, event: ObjCObject):
        var p = event_point(event.ptr())
        paint_at(p.x, p.y, event_has_shift(event.ptr()))

    def mouseDragged_(self, event: ObjCObject):
        ...

    def rightMouseDown_(self, event: ObjCObject):
        var p = event_point(event.ptr())
        paint_at(p.x, p.y, True)

    def acceptsFirstResponder(self) -> Bool:
        return True

    def keyDown_(self, event: ObjCObject):
        handle_key(event.ptr())
```

`NSView` as the first base makes it a subclass. Six methods, six selectors, every encoding from the SDK.

`acceptsFirstResponder` has no underscore and no arguments, so it is the nullary selector `acceptsFirstResponder`. Returning `True` is what makes the view eligible for key events at all — without it `keyDown_` never fires and nothing indicates why.

Note also what is *not* in the class body. `handle_key` and `event_point` are free functions, and the source says why:

> Key handling, kept a free function so the class body stays a list of selectors rather than a wall of logic.

That is a good instinct. A Cocoa class is an interface to the framework; the logic behind it does not have to live inside it.

State lives in globals, for now

```
comptime g_alive = named_global["life.alive", Int]   # UInt8*  per cell 0/1
comptime g_next = named_global["life.next", Int]     # UInt8*  scratch
comptime g_age = named_global["life.age", Int]       # UInt16* generations survived
comptime g_frame = named_global["life.frame", Int]   # UInt32* BGRA pixels
comptime g_running = named_global["life.running", Int]
comptime g_speed = named_global["life.speed", Int]
```

Fourteen of these. They are not there because globals are nice; they are there because **a class cannot hold fields yet**. When that lands, most of this becomes ordinary members with real construction and destruction. Until then, this is the pattern, and the `life.` prefix on every key is the discipline that keeps two subsystems from colliding.

`main`, in order

Load AppKit first

```
def main() raises:
    if not load_framework["AppKit"]():
        raise Error("could not load AppKit")
```

Nothing linked AppKit into a JIT-run process, so without this the `NSApplication` lookup is nil and every message to nil quietly does nothing: the app starts and exits having drawn nothing.

Allocate outside Mojo

```
    g_alive()[] = alloc_zeroed(CELLS, 1)
    g_frame()[] = alloc_zeroed(PIXELS, 4)
```

Mojo destroys a value at its **last use**, not at end of scope, so a `List` whose `unsafe_ptr()` you stash in a global is freed immediately and every stored pointer dangles — surfacing much later as a corrupted allocator, nowhere near the cause. Owning the memory outside Mojo makes the lifetime explicit and correct.

Instantiate the classes

```
        # Instantiating a class is what registers it, so the delegate exists
        # in the runtime by the time it is handed over.
        var delegate = ObjCObject(LifeDelegate().__objc_id)
        _ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](
            app, delegate.ptr()
        )
```

```
        var view_instance = ObjCObject(LifeView().__objc_id)
        var actions = ObjCObject(LifeActions().__objc_id)
        _ = external_call["objc_retain", P](actions.ptr())
```

Three lines where the old version had three builders, six `add_method` calls and two hand-written encodings.

Two details worth taking away.

Instantiation is registration. There is no separate register step, so the class is in the runtime by the time you hand the instance to AppKit.

Retain what Cocoa holds by bare pointer. `actions` is the timer's target and must outlive this scope; the explicit `objc_retain` is correct for an object that lives as long as the application.

A difference from the builder path

```
        # `LifeView()` is already allocated and initialised -- that is what
        # instantiating a class does -- so the frame is set rather than passed
        # to initWithFrame:.
        var view = view_instance
        _ = msg_send[ObjCObject, "NSView", "setFrame:"](view, frame)
```

Worth noticing because it changes the code you write. With `ObjCClassBuilder` you got a class back and did `alloc` then `initWithFrame:` yourself. `LifeView()` has already done both, so you set the frame afterwards instead.

The timer

```
        _ = msg_send[
            ObjCObject, "NSTimer",
            "scheduledTimerWithTimeInterval:target:selector:userInfo:repeats:",
            is_class=True,
        ](
            ObjCClass.lookup["NSTimer"]().as_object(),
            Float64(1.0 / 60.0),
            actions.ptr(),
            sel["lifeTick:"]().ptr(),
            actions.ptr(),
            Bool(True),
        )
```

`sel["lifeTick:"]()` is the one place the selector appears as a value rather than as a `msg_send` parameter — target/action wants the `SEL` itself.

`Float64(1.0 / 60.0)` and not `1.0 / 60.0`: the interval travels in a float register, and the register-file check would have caught an integer literal.

Into the run loop

```
    var app2 = msg_send[
        ObjCObject, "NSApplication", "sharedApplication", is_class=True
```

```
    ](ObjCClass.lookup["NSApplication"]().as_object())
    _ = msg_send[ObjCObject, "NSApplication", "run"](app2)
```

Outside the `with` block, so the setup pool drains before the loop begins; the loop keeps its own pools per event cycle. `app` was bound inside the block that has now ended, hence `app2`.

`run` does not return.

What the drawing does

The parts this chapter has skipped are ordinary Mojo. `evolve()` runs the Life rules over the two cell buffers; `render()` writes BGRA pixels into `g_frame`; `present()` blits that into a `CAMetalLayer` drawable. The Metal work uses `send` rather than `msg_send`, because a `MTLDevice` is a protocol-typed object whose concrete class is not something you can name:

```
        var queue = send[ObjCObject, "newCommandQueue"](display_dev)
```

What to take from it

The Cocoa-facing surface of a 644-line application is three class declarations totalling about thirty lines, and every selector and encoding in them was checked at compile time. The rest is Mojo.

That ratio is the point of the whole design.

CocoaMojo Reference Manual

Look-up material, organised by what you are trying to find. If you are learning rather than checking, start with the Programmer's Guide.

Section	Contents
1. Language reference	The dialect this frozen compiler accepts: keywords, conventions, declarations, and a complete list of what was removed
2. std.objc	Every exported type and function, with signatures
3. cocoakb queries	Every compile-time metadata query, its parameters and its result
4. Diagnostics	The compile errors this layer produces, what each means, and how to fix it

Version

Everything here describes the compiler at the fork point of MojoCocoa, Mojo 1.1.0. The fork does not track upstream, so this reference does not go stale in the usual way — it describes one compiler that does not change.

Facts were taken from the source tree: `mojo/stdlib/std/objc/`, `mojo/stdlib/std/sys/_cocoakb.mojo`, `KGEN/lib/MojoParser/`, and the verification spikes under `spikes/`.

Notation

Parameters appear in square brackets and are resolved at compile time. Arguments appear in parentheses. A parameter with `= value` is defaulted.

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R
```

Throughout, `P` abbreviates `OpaquePointer[MutUntrackedOrigin]`, following the convention used in the standard library itself.

1. Language reference

The dialect accepted by this frozen compiler. Where it differs from published Mojo documentation, this document describes the compiler.

Declarations

Functions

Two keywords, with different contracts.

```
def name(arg: T, ...) -> R:          # ordinary Mojo function
def name(arg: T) raises -> R:
def name[param: T](arg: T) -> R:

fn name(a: P, b: P) -> Bool:         # foreign-callable: thin, C ABI, non-raising
```

`raises` marks a `def` that can raise. `abi("C")` on a `def` gives the C calling convention and belongs before the return arrow; `fn` implies it.

fn — the foreign-callable function. Thin (no captures), non-raising, C ABI, all parameter and return types ABI-classifiable. Exactly the Objective-C `IMP` contract. Callable from Mojo as an ordinary function. May not be marked `raises` or `async`.

Bindings

```
var x = expr        # mutable binding
let x = expr        # immutable binding, function body only
comptime X = expr   # compile-time binding
```

`let` is immutable in the *binding*, not the object: `let win = ...` still permits `setTitle:`. It is general — it applies to values as well as reference-counted objects. Ownership semantics for Cocoa objects live in `ObjCRef`, not in the keyword. There is no field or file-scope `let`.

Function types

```
fn(T1, T2, /) -> R                   # sugar for the line below
def(T1, T2, /) thin abi("C") -> R
def[w: Int](Int) thin -> None where (w > 0, "width must be positive")
```

`/` ends the positional-only parameters. `thin` means no captured context. A `thin` function type may carry trailing `where` clauses constraining the parameters it declares; the clause binds to the innermost function type, so a declaration-level `where` after a function-type result needs that result parenthesised:

```
def make[n: Int]() -> (def() thin -> None) where n > 0: ...
```

Binding a constrained function to a function type that declares no matching `where` clause is an error. Passing an unconstrained function where a constrained type is expected is allowed and free.

Compile-time bindings

```
comptime NAME = expression
comptime Alias = SomeType[Param]
```

`alias` still parses but is deprecated: `'alias' is deprecated; use 'comptime'`.

Compile-time control flow

```
comptime if cond:
    ...
else:
    ...

comptime for i in range(n):
    ...

comptime assert condition, "message"
```

`@parameter if` and `@parameter for` still parse but are deprecated. The `@parameter` decorator on parametric closures is now `@__parameter`.

Structs

```
@fieldwise_init
struct Name(Trait1, Trait2):
    var field: T

    def __init__(out self, ...):
    def method(self) -> T:
    def mutator(mut self):
    def consume(deinit self) -> T:
    def __deinit__(deinit self):
```

Struct parameters are declared in brackets after the name:

```
struct ObjCClassBuilder[superclass: StaticString = "NSObject"]:
```

`__new__` is not supported on structs; use `__init__`.

Argument conventions

Convention	Meaning
(omitted)	Immutable borrow. The default.

`imm`	Immutable borrow, written explicitly.
`mut`	Mutable borrow.
`out`	Uninitialised result the callee must initialise.
`var`	By value; callee owns it.
`deinit`	Consumes the value, ending its lifetime.

`read` is the deprecated spelling of `imm` and warns with a FixIt. `inout`, `borrowed` and `owned` do not exist.

`imm`, `mut`, `out`, `deinit` and `where` are **contextual** — soft identifiers, not reserved words — so they remain usable as ordinary names. `var` is a reserved keyword.

Function types do not accept `deinit`; use `var` there.

The transfer sigil `^` moves a value into a consuming position:

```
var cls = builder^.register()
```

Reserved words

The compiler reserves 64 keywords. Most you will use; a handful are lexed and classified but have no parse handling at all, and a block of double-underscore names are compiler internals.

Ordinary

```
_        alias    and      as       assert   async    await    break
comptime continue def      elif     else     except   finally  fn
for      from     if       import   in       is       let      not
or       pass     raise    ref      return   struct   trait    try
var      while    with     lambda
```

`alias` warns and is superseded by `comptime`; `fn` and `let` have the cocoa-mojo meanings described above.

Reserved but not implemented

```
del      global   match    case     nonlocal   yield
```

Lexed and classified as statement keywords and then never parsed, so they are unavailable as identifiers without buying you anything. There is no pattern matching, no `del`, no generators, and no `global`/`nonlocal`.

`class` was in this list and no longer is — see below.

`class`

Declares an **Objective-C class**, not a Mojo one.

```
class Name(Superclass, Protocol, ...):
    def method_(self, arg: T) -> R: ...     # may raise; the boundary catches
    fn strict_(self, arg: T): ...           # fn's contract, unchanged
    @objc("real:selector:")
    def renamed(self, a: T, b: U): ...
```

The first base is the superclass and defaults to `NSObject`; the rest are protocols, adopted through `class_addProtocol`. Method names map to selectors by turning a trailing `_` into `:`, with the underscore count required to match the argument count. A **leading** underscore marks a method as Mojo's own, never exposed to the runtime.

Encodings are looked up from the SDK when the selector exists on the superclass chain — a disagreement is a compile error quoting the database — and derived from the Mojo signature otherwise.

`ClassName()` registers the class if needed, then allocs and inits; registration is idempotent. `__objc_id` is the raw `id`. A class reference is register-passable, retains on copy and releases on destruction.

Not in this version: fields (`var` members), nested classes, class-level `comptime` parameters, Mojo-trait conformances.

Compiler internals

```
__comptime_assert       __extension              __functions_in_module
__generator_type        __get_address_as_owned_value
__get_address_as_uninit_lvalue                   __get_current_function_name
__get_litref_as_mvalue  __get_mvalue_as_litref   __is_run_in_comptime_interpreter
__mlir_region           __mojo_crash             __struct_field_ref
conforms_to             origin_of                type_of
```

`conforms_to`, `origin_of` and `type_of` are the three you will actually write — they appear in `where` clauses and `comptime assert`s throughout the standard library. `__comptime_assert` is deprecated in favour of `comptime assert`. The rest are the compiler talking to itself.

Contextual, not reserved

```
imm      mut      out      deinit   where    read
```

These are soft identifiers: they mean something in a signature and remain usable as ordinary names everywhere else. `read` is deprecated in favour of `imm`.

Origins

Spelling	Removed alias
`ImmOrigin`	`ImmutOrigin`
`ImmUnsafeAnyOrigin`	`ImmutUnsafeAnyOrigin`

`ImmStaticOrigin`	`StaticConstantOrigin`
`UntrackedOrigin`	`ExternalOrigin`
`MutUntrackedOrigin`	`MutExternalOrigin`
`ImmUntrackedOrigin`	`ImmutUntrackedOrigin`, `ImmutExternalOrigin`

Also in use: `MutAnyOrigin`, `MutOrigin`.

Pointers

`Pointer[T, Origin]` is the type. `OpaquePointer[Origin]` is the untyped one. `UnsafePointer` exists but is deprecated in favour of `Pointer`.

Removed aliases: `MutUnsafePointer`, `ImmUnsafePointer`, `ImmutUnsafePointer`, `ImmutOpaquePointer`, `ImmutPointer`, `OptionalUnsafePointer`. Use `MutPointer`, `ImmPointer`, `ImmOpaquePointer`, `OptionalPointer`.

Construction and use:

```
Pointer[UInt8, MutUntrackedOrigin](unsafe_from_address=addr)
OpaquePointer[MutUntrackedOrigin](unsafe_from_address=addr)
p[]                                  # dereference
Pointer(to=value).unsafe_bitcast[T]()
p.bitcast[T]()
```

Renamed methods: `unsafe_as_noalias()`, `unsafe_deinit_pointee()`, `unsafe_deinit_pointee_with()`, `unsafe_write()`, `unsafe_write_move_from()`. `Pointer.type` is now `Pointer.T`.

Renamed free functions: `unsafe_memcpy`, `unsafe_memset`, `unsafe_memset_zero`, `unsafe_uninit_move_n`, `unsafe_uninit_copy_n`, `unsafe_destroy_n`, `unsafe_memcmp`.

Traits in common use

`AnyType`, `Copyable`, `Movable`, `Deinitable`, `TrivialRegisterPassable`.

`ImplicitlyDestructible` and `ImplicitlyDeletable` were renamed to `Deinitable`. `@explicit_destroy` is no longer required. The `register_passable` and `escaping` function effects are no longer supported.

Decorators

`@fieldwise_init`, `@always_inline`, `@staticmethod`, `@export("name")`, `@deprecated(...)`, `@__parameter`.

MLIR escape hatches

Used by the standard library where no surface syntax exists:

```
__mlir_op.`pop.global_alloc`[...]()
__mlir_op.`pop.extern_ptr_symbol`[...]()
__mlir_attr[`#kgen.param.expr<...>`]
```

These are internal, carry no stability guarantee, and are documented here only so that reading the standard library is possible.

Modules

Directories may have namespace semantics: one directory name may resolve across distinct locations on disk that share the name.

Importing same-named functions from different modules to form one overload set is an error. Intra-package access without an explicit `import` is an error.

`.mojopkg` is gone; the package format is `.mojoc`.

Removed library APIs

Removed	Replacement
`InlineArray`	`Array`
`SIMD.size`, `Array.size`, `TypeList.size`, `SIMDSize`	`length`, `SIMDLength`
`String.as_string_slice()`	`StringSpan(s)`
`String.set_byte_length()`	*(internal, none)*
`as_immutable()`, `get_immutable()`	`as_imm()`
`ImmutSpan`	`ImmSpan`
`ConditionalType`, `std.utils.type_functions`	`T if cond else U`
`trait_downcast()`	`conforms_to(...)` in `where` or `comptime assert`
`memcmp`	`unsafe_memcmp`
`List.steal_data()`, `OwnedPointer.steal_data()`	`unsafe_take_allocation()`
`OwnedPointer.take()`	`into_inner()`
`Variant.take()`, `Variant.unsafe_take()`	`unwrap()`, `unsafe_unwrap()`
`b64decode(validate=...)`	always validates; drop the parameter
`std.gpu.profiler`, `ProfileBlock`	`perf_counter_ns()`, external profiler
`benchmark.run[func]()`	`run(f)`
`AnyCoroutine`, `Coroutine`, `RaisingCoroutine`	*(unnameable; async def still works)*

async task API in `std.runtime.asyncrt`	`initialize_runtime()`, `parallelism_level()` from `std.runtime`

2. std.objc

The complete exported surface. Throughout, `P` abbreviates `OpaquePointer[MutUntrackedOrigin]`.

```
from std.objc import (
    ObjCClass, ObjCObject, SEL, sel, msg_send, send, autoreleasepool,
    load_framework,
    ObjCRef, ObjCWeakRef,
    NSString, nsstring, extern_object, ns_to_string,
    msg_send_raising, msg_send_raising_check,
    ObjCClassBuilder, IMP0, IMP1, IMP0Bool, IMP1Bool, IMP2,
    new_instance, named_global, sel_dynamic,
)

# GCD lives in its own module
from std.objc.dispatch import (
    Semaphore, async_f, global_queue, main_queue, sync_f, with_block,
)
```

Runtime handles

SEL

A registered Objective-C selector — an interned string pointer.

Member	Signature
`ptr`	`def ptr(self) -> P`

Conforms to `TrivialRegisterPassable`.

ObjCClass

A handle on a class object.

Member	Signature
`lookup`	`@staticmethod def lookup[name: StaticString]() -> ObjCClass`
`is_nil`	`def is_nil(self) -> Bool`
`as_object`	`def as_object(self) -> ObjCObject`

`lookup` calls `objc_getClass`. A nil result means the framework is not loaded.

ObjCObject

A borrowed `id`.

Member	Signature

`is_nil`	`def is_nil(self) -> Bool`
`addr`	`def addr(self) -> Int`
`ptr`	`def ptr(self) -> P`

Construct directly from an address with `ObjCObject(Int(raw_pointer))`.

Selectors

`sel`

```
def sel[name: StaticString]() -> SEL
```

Registers the selector once and caches the result in a per-selector global slot, deduplicated by name in the KGEN lowering. After the first send the cost is one load and a null check.

`sel_dynamic`

```
def sel_dynamic(name: StaticString) -> P
```

Registers a selector by name at run time, returning the raw pointer. Accepts custom selectors the SDK does not know. Use when calling `class_addMethod` directly.

Message sending

`msg_send`

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R
```

Sends `selector` to `obj`, returning `R`.

Parameter	Meaning
`R`	Return type; must be register-passable
`cls`	Class the selector is looked up on; inheritance-resolved
`selector`	The selector, colons included
`is_class`	`True` for a + method
`Ts`	Argument types, inferred

Compile-time checks: the selector must exist on `cls` or a superclass; the argument count must equal the selector's declared count; each argument must be in the register file the ABI expects, for the cases that can be determined certainly; and the `@encode` signature must be modelable.

send

```
def send[R: AnyType, selector: StaticString, *Ts: AnyType](
    obj: ObjCObject, *args: *Ts
) -> R
```

Sends to a **protocol-typed** receiver whose concrete class is unknown at compile time — every Metal object, every delegate. The dispatch stub and argument classes come from the database keyed by selector alone.

Argument count and register-file checks still apply. Receiver-class verification does not.

Ownership

ObjCRef

An owning handle on one +1 reference. Conforms to `Movable`; deliberately not implicitly `Copyable`.

Member	Signature	Notes
init	`def __init__(out self, *, adopt: ObjCObject)`	Takes ownership of an object already at +1
init	`def __init__(out self, *, retain: ObjCObject)`	Adds a +1 to a borrowed object
`copy`	`def copy(self) -> Self`	Explicit retain
`object`	`def object(self) -> ObjCObject`	Borrowed handle, valid for this ref's lifetime
`is_nil`	`def is_nil(self) -> Bool`	
`autorelease`	`def autorelease(deinit self) -> ObjCObject`	Hands to the current pool; consumes the ref
deinit	`def __deinit__(deinit self)`	Releases if not nil

Use `adopt=` after `alloc`, `new`, `copy`, `mutableCopy`; `retain=` otherwise.

Backed by the ARC entry points `objc_retain`, `objc_release`, `objc_autorelease`, so it interoperates with ARC code.

autoreleasepool

A scoped pool.

```
with autoreleasepool():
```

```
    ...
```

Pushes on `__enter__`, pops on `__exit__`, and guards against a double pop — popping a token twice corrupts the pool page.

Foundation

NSString

A leak-safe owning wrapper. Conforms to `Movable`.

Member	Signature
init	`def __init__(out self, *, adopt: ObjCObject)`
init	`def __init__(out self, text: String)`
`object`	`def object(self) -> ObjCObject`
`length`	`def length(self) -> Int`
`to_string`	`def to_string(self) -> String`
`equals`	`def equals(self, other: NSString) -> Bool`
`appending`	`def appending(self, other: NSString) -> NSString`

`length` is UTF-16 code units, matching `-[NSString length]`.

nsstring

```
def nsstring(s: String) -> ObjCObject
```

An **autoreleased** `NSString`. For handing to AppKit setters, which retain. Use inside an autorelease pool.

extern_object

```
def extern_object[name: StaticString]() -> ObjCObject
```

The object held in an extern Cocoa constant, such as `NSForegroundColorAttributeName`. These are globals whose address the linker resolves, not compile-time values; this takes a link-time reference to the data symbol and loads the pointer out of it.

Defining classes

IMP type aliases

Alias	Signature
IMP0	fn(P, P, /) -> None
IMP1	fn(P, P, P, /) -> None
IMP0Bool	fn(P, P, /) -> Bool
IMP1Bool	fn(P, P, P, /) -> Bool
IMP2	fn(P, P, P, P, /) -> None

The first two arguments are always `self` and `_cmd`.

ObjCClassBuilder

Superseded by the class keyword for new code; retained as the lower-level mechanism and as the escape hatch for method shapes `class` does not cover.

```
struct ObjCClassBuilder[superclass: StaticString = "NSObject"]
```

Member	Signature
init	def __init__(out self, name: String)
add_method	def add_method[selector: StaticString, encoding: StaticString = ""](mut self, imp: IMPn)
register	def register(deinit self) -> ObjCClass

`add_method` is overloaded across the five IMP shapes. When `encoding` is omitted the SDK's `@encode` for the selector is used, with frame offsets stripped; an unknown selector with no `encoding` is a compile error.

`register` consumes the builder, so call it as `builder^.register()`.

new_instance

```
def new_instance(cls: ObjCClass) -> ObjCObject
```

`+[cls new]` — an owned (+1) instance. Retain it explicitly if it must outlive the enclosing scope while Cocoa holds a bare pointer to it.

named_global

```
def named_global[name: StaticString, T: AnyType]() -> Pointer[T, MutUntrackedOrigin]
```

A zero-initialised process global of type `T`, shared by name across every call site, because KGEN deduplicates the global. The standard answer to callbacks having no closure.

```
comptime g_running = named_global["life.running", Int]
```

```
g_running()[] = 1
```

Frameworks

load_framework

```
def load_framework[name: StaticString]() -> Bool
```

`dlopen` of `/System/Library/Frameworks/<name>.framework/<name>` with `RTLD_NOW`. Returns whether the handle is non-null. Idempotent and cheap after the first call.

Foundation arrives in every process; AppKit does not unless the binary linked it, which a `mojo run` process did not. Without this, `ObjCClass.lookup` for an AppKit class returns nil and every message to it silently no-ops.

Errors

msg_send_raising

```
def msg_send_raising[
    cls: StaticString, selector: StaticString,
    T0: AnyType, ..., is_class: Bool = False,
](obj: ObjCObject, a0: T0, ...) raises -> ObjCObject
```

Sends an object-returning `...error:` selector and raises when the result is nil. **The trailing error argument is created and appended by the wrapper — pass the message arguments without it.**

msg_send_raising_check

```
def msg_send_raising_check[
    cls: StaticString, selector: StaticString,
    T0: AnyType, ..., is_class: Bool = False,
](obj: ObjCObject, a0: T0, ...) raises
```

The `BOOL` convention: raises when the result is `NO`. Returns nothing. Same error-slot handling.

Both are declared at explicit arities up to five leading arguments, because nothing may follow an unpacked `*args`. The raised `Error` carries the `NSError`'s `localizedDescription`, domain and code, copied into a Mojo `String`. An API that returns failure without writing an error produces a message saying exactly that.

Weak references

ObjCWeakRef

A zeroing weak reference. Conforms to `Movable`.

Member	Signature	Notes
init	`def __init__(out self, target: ObjCObject)`	Registers via `objc_initWeak`; a nil target is legal
`load`	`def load(self) -> ObjCRef`	The object at +1 if alive, else a nil `ObjCRef`
`copy`	`def copy(self) -> Self`	An independent registration
deinit	`def __deinit__(deinit self)`	`objc_destroyWeak` plus `free`

The registration slot is heap-allocated — one pointer-sized `malloc` per weak reference — because the runtime tracks weak references by the *address of the slot* and Mojo values move. A struct field would leave the runtime pointing into a dead frame after any move.

`load()` goes through `objc_loadWeakRetained` and therefore returns an owned reference, so the object cannot be deallocated between the nil-check and the use.

Use this wherever Cocoa expects weakness — delegates and observer targets — because an `ObjCRef` there is a retain cycle.

Strings, the other direction

ns_to_string

```
def ns_to_string(s: ObjCObject) -> String
```

`NSString` to Mojo `String`, by copy, UTF-8. The result does not depend on the pool owning the `NSString`.

std.objc.dispatch

Grand Central Dispatch. Imported directly, not re-exported from `std.objc`.

Name	Signature	Notes
`DispatchFn`	`fn(_RawPtr) -> None`	The work-function contract
`global_queue`	`def global_queue() -> _RawPtr`	Default-priority concurrent queue
`main_queue`	`def main_queue() -> _RawPtr`	`&_dispatch_main_q`, as the C macro takes it
`sync_f`	`def sync_f(queue, context, work: DispatchFn)`	Runs before returning
`async_f`	`def async_f(queue, context, work: DispatchFn)`	Returns immediately
`Semaphore`	`struct Semaphore(Movable)`	`.wait()`; one wait per expected

		signal
`with_block`	`def with_block[work: fn() -> None](queue, *, wait: Bool)`	Builds a real ObjC block around an `fn`

`with_block` works because a thin `fn` is exactly a global block — `_NSConcreteGlobalBlock`, no captures, so no copy/dispose helpers. The `wait=True` path uses a stack literal; `wait=False` lets the runtime `Block_copy` the 32 bytes off the frame.

Capturing closures as heap blocks are not implemented.

Method type encodings

The strings `class_addMethod` expects, as returned by the database with frame offsets stripped.

Encoding	Meaning
`v@:`	`void method(id self, SEL _cmd)`
`v@:@`	`void method(id self, SEL _cmd, id arg)`
`c@:`	`BOOL method(id self, SEL _cmd)`
`c@:@`	`BOOL method(id self, SEL _cmd, id arg)`
`@@:`	`id method(id self, SEL _cmd)`

Single-character type codes: `v` void, `c` `BOOL` (a signed char), `@` object, `:` selector, `i` int, `q` long long, `Q` unsigned long long, `d` double, `f` float, `*` C string, `^` pointer to.

`BOOL` is `c`, not `B`. This is the encoding most often written wrongly by hand.

3. cocoakb queries

Compile-time queries against `cocoa.sqlite`. Every one resolves to a constant before code generation, so a name the metadata does not know is a compile error rather than a wrong answer.

```
from std.sys._cocoakb import cocoakb_struct_size, cocoakb_field_offset
```

The module is underscore-prefixed because it is compiler-adjacent, not because it is off limits — the verification spikes import it directly.

Layout

`cocoakb_struct_size`

```
def cocoakb_struct_size[name: StaticString]() -> Int
```

Size in bytes of a Cocoa struct. `name` is spelled as the runtime spells it, for example `"CGRect"`.

`cocoakb_struct_align`

```
def cocoakb_struct_align[name: StaticString]() -> Int
```

Alignment in bytes.

`cocoakb_field_offset`

```
def cocoakb_field_offset[type_name: StaticString, field: StaticString]() -> Int
```

Byte offset of a field within a struct. This is what makes a declaration checkable rather than merely plausible.

```
comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()
comptime assert cocoakb_field_offset["CGRect", "size"]() == 16
```

Constants

`cocoakb_enum_value`

```
def cocoakb_enum_value[name: StaticString]() -> Int
```

The value of an enum member, from BridgeSupport. Signed, so a value that must sign-extend as a pointer does behaves correctly, and a flag mask narrowed by the caller keeps its bits either way.

```
comptime assert cocoakb_enum_value["NSUTF8StringEncoding"]() == 4
```

cocoakb_constant_type

```
def cocoakb_constant_type[name: StaticString]() -> StaticString
```

The declared type encoding of an extern-symbol constant, for example `"@"` for an object. Constants like `NSFontAttributeName` are runtime addresses, not compile-time values; this reports the type, and `extern_object` fetches the value.

Classes and methods

cocoakb_superclass

```
def cocoakb_superclass[name: StaticString]() -> StaticString
```

cocoakb_method_encoding

```
def cocoakb_method_encoding[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

The verbatim `@encode` signature, inheritance-resolved: the lookup walks the superclass chain, so asking `NSMutableString` about `length` finds `NSString`'s definition. Returns the raw form including frame offsets, for example `"Q16@0:8"`.

cocoakb_msgsend_variant

```
def cocoakb_msgsend_variant[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

Which `objc_msgSend` entry point the send must use. Returns `"objc_msgSend"`, `"objc_msgSend_stret"`, or `"objc_msgSend_fpret"`; `"?"` marks a signature the ABI classifier could not model, and callers assert against it.

On arm64 the answer is always `"objc_msgSend"`. AAPCS64 has neither `_stret` nor `_fpret` — an aggregate return travels in `x0`–`x1`, in `v0`–`v3` when it is a homogeneous float aggregate, or through the `x8` indirect-result register. The query is retained because its other job — rejecting unmodelable signatures — still matters.

cocoakb_method_ret_class

```
def cocoakb_method_ret_class[
```

```
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

The AAPCS64 return classification, in the token vocabulary below.

cocoakb_method_arg_classes

```
def cocoakb_method_arg_classes[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

Comma-joined classifications of the arguments beyond `self` and `_cmd`. An empty string means the selector takes none; `"g"` means one; `"g,g"` two.

Selector-keyed queries

For protocol-typed receivers whose concrete class is unknown at compile time.

cocoakb_selector_variant

```
def cocoakb_selector_variant[selector: StaticString]() -> StaticString
```

The dispatch variant, taken from any class implementing the selector. An empty result or `"?"` means no class in the metadata implements it.

cocoakb_selector_arg_classes

```
def cocoakb_selector_arg_classes[selector: StaticString]() -> StaticString
```

cocoakb_selector_encoding

```
def cocoakb_selector_encoding[selector: StaticString]() -> StaticString
```

The `@encode` signature by majority across implementing classes, for example `"v24@0:8@16"`. Used by `ObjCClassBuilder` to type a Mojo method when defining a class at run time.

POSIX

The same database describes the C library.

Query	Returns
`cocoakb_posix_sig[name]()`	The full C type as clang reports it, e.g. `"int (const char *, int, ...)"`
`cocoakb_posix_ret_class[name]()`	AAPCS64 return classification

`cocoakb_posix_arg_classes[name]()`	AAPCS64 argument classifications

Provenance

cocoakb_db_hash

```
def cocoakb_db_hash() -> StaticString
```

The SHA-256, lowercase hex, of the database this compilation consulted. A binary can record exactly which metadata revision produced it, which is the reproducibility pin and also how you notice a stale database after a macOS update.

The AAPCS64 token vocabulary

These tokens come from `derive_method_abi.py` in CocoaBaseMCP. They are **not** the SysV eightbyte vocabulary, and reading them as if they were puts a homogeneous float aggregate in the wrong register file.

Token	Meaning	Register file
`v`	void	—
`g`	general-purpose register	integer
`f`	scalar float	float, `v0–v7`
`h2`, `h3`, `h4`	homogeneous float aggregate of k members	float, k consecutive `v` registers
`i1`, `i2`	small integer struct	integer
`b`	by-value pointer	integer
`x`	indirect result via `x8`	—
`s`	struct	integer
`m`	memory	—
`?`	unmodelable; a compile error	—

A value goes in a `v` register when it is a scalar float or a homogeneous float aggregate. Everything else goes in the integer file. Testing whether every byte of a token is `f` would be the SysV reading and would misclassify an HFA.

Worked examples from the verification spike:

Query	Result
`cocoakb_method_ret_class["NSString", "length"]()`	`"g"` — one GPR
`cocoakb_method_ret_class["NSValue", "rectValue"]()`	`"h4"` — `CGRect` in `v0–v3`

`cocoakb_method_encoding["NSMutableString", "length"]()`	`"Q16@0:8"`

4. Diagnostics

The compile errors this layer produces, what each one means, and what to do. Messages are quoted from the source that emits them.

Language-level

`'fn' declares a foreign-callable (C ABI, non-raising) function in cocoa-mojo and may not be marked 'raises'; use 'def' for an ordinary Mojo function`

`fn` is the foreign-callable function. The C boundary has no error channel, so a raising `fn` cannot exist. Comes with a FixIt replacing `fn` with `def`.

This is also the migration message: code from the era when `fn` was a general strict function (`fn main() raises`) gets told what changed rather than a bare contract violation.

`'fn' declares a foreign-callable function and cannot be 'async'; use 'def'`

Same contract, same remedy.

`'let' declares an immutable binding inside a function body; use 'var' for a field or module value`

`let` has no field or file-scope form. Use `var` there.

Reassigning a `let`

Rebinding a `let` is a compile error. The object remains mutable — this is about the binding, not the value. See the language chapter.

`'alias' is deprecated; use 'comptime'`

A warning, not an error. `alias` still parses.

`'__new__' is not supported on structs; use '__init__' instead`

`the 'register_passable' function effect is no longer supported`

`the 'escaping' function effect is no longer supported`

`implicit deletion. '@explicit_destroy' is no longer required.`

Remove the decorator; implicit deletion is now the default.

`std.objc` compile-time assertions

These come from `comptime assert` inside `msg_send`, `send` and `ObjCClassBuilder`, so they fire during elaboration and name the problem precisely.

Unknown or unmodelable signature

```
std.objc: 'SELECTOR' on CLASS has an @encode signature the ABI classifier
could not model, so std.objc cannot pick a dispatch stub for it. Call it by
hand with a checked external_call if you know the layout.
```

`cocoakb_msgsend_variant` answered `"?"`. Rare, and the escape hatch is the one the message names: call it yourself with `external_call`, having worked out the layout.

Wrong argument count

```
std.objc: 'SELECTOR' on CLASS takes N argument(s), but M were passed.
```

Count the colons in the selector. `stringWithUTF8String:` takes one; `initWithContentRect:styleMask:backing:defer:` takes four. This is the check that prevents reading an unset argument register.

Wrong register file — float where an integer is expected

```
std.objc: argument I of 'SELECTOR' on CLASS is a float, but the ABI expects an
integer/pointer register here. Check the argument type.
```

Wrong register file — integer where a float is expected

```
std.objc: argument I of 'SELECTOR' on CLASS is an integer, but the ABI expects
a float register here. Pass a Float32/Float64.
```

The usual cause is an integer literal where a `CGFloat` is wanted. Write `Float64(0)` rather than `0`. These checks fire only where the classification is certain, so an absence of error is not proof of correctness for struct arguments.

Selector implemented by nothing

```
std.objc: no class in the metadata implements selector 'SELECTOR', so its
dispatch ABI is unknown. Check the selector spelling.
```

From `send`. Either the selector is misspelled, or the framework declaring it was not scanned when the database was built.

Unknown selector encoding when defining a class

Raised by `ObjCClassBuilder.add_method` when `encoding` is omitted and the SDK does not know the selector. Supply it:

```
b.add_method["myCustomAction:", encoding="v@:@"](handler)
```

cocoakb query failures

A name the database does not contain fails the query itself. The `must_fail.mojo` spike is exactly this:

```
comptime bogus = cocoakb_struct_size["NSDefinitelyNotAStruct"]()
```

The build fails. It does not return zero, and it does not return a plausible size.

If a name you believe is real fails, work through these in order.

Is the database built? `python3 ../CocoaBaseMCP/build.py`.

Is the compiler pointed at it? `MODULAR_MOJO_MAX_COCOAKB_PATH` must name an existing `cocoa.sqlite`.

Is it stale? Rebuild after a macOS update. `cocoakb_db_hash()` tells you which revision a compilation used.

Is the spelling the runtime's? The database is built from the live Objective-C runtime and BridgeSupport, so it holds the runtime's names, not a header's typedef.

Runtime failures the compiler cannot catch

Three things remain yours.

A nil class. `ObjCClass.lookup` returns nil when the framework is not loaded. Every subsequent message to it does nothing, silently. Check `is_nil()` once at startup.

Use after the pool drains. An object handed to a pool by `autorelease()` is valid only until that pool exits. Nothing checks this.

An object released while Cocoa still holds it. Typically an `ObjCRef` that went out of scope, or a `new_instance` result never retained, where Cocoa keeps a bare pointer. The symptom is a crash inside `objc_msgSend` with a plausible-looking receiver.

Symptoms with no error at all

Symptom	Cause
Window appears but ignores the keyboard	`acceptsFirstResponder` missing or returning `False`
Window cannot be focused, no	`setActivationPolicy:` not called with `0`

Dock icon	
Callback never fires	Selector added under a different spelling, or `register()` never ran
`respondsToSelector:` returns `False`	Same as above; check the selector string character by character
Allocator corruption far from any Cocoa call	A `List` pointer stashed after its last use; allocate outside Mojo instead
`autorelease pool page corrupted`	A pool token popped twice
Windowed app starts and exits with no window and no message	`load_framework["AppKit"]()` was not called; every message to the nil class silently no-ops
A crash when assigning into a `var` declared but not initialised	Pre-existing: assigning into an uninitialised `var` of a type with `__deinit__` destroys garbage first. Bind in a helper scope and return instead